



FORMATION INFORMATIQUE - SUITE

HTML et CSS - La suite

Semantique, cascade, variables, Grid,
images et a11y : la suite apres les bases.

DanielCraft - Livre debutant

Sommaire

Clique un chapitre ou un sous-chapitre pour y aller.

1. Chapitre 1 - Rappel rapide et carte du livre

- [Ce que tu gardes en tete](#)
- [Ce que ce n'est pas](#)
- [La carte du livre](#)
- [Un fil rouge](#)
- [Ce dont tu as besoin](#)
- [Comment lire ce livre](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)

2. Chapitre 2 - HTML semantique : donner un sens a la page

- [Pourquoi ca compte vraiment](#)
- [Les balises de structure que tu vas utiliser](#)
- [Une page blog, version semantique](#)
- [Landing simple](#)
- [Carte produit](#)
- [Ce que ce n'est pas](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)

3. Chapitre 3 - Cascade et specificite : qui gagne ?

- [L'ordre compte](#)
- [Qu'est-ce qui pese plus ?](#)
- [Classes plutot que ID pour le style](#)
- [Selecteurs composes](#)
- [Heritage \(rapide\)](#)
- [Conflit classique sur une carte](#)
- [!important, le extincteur](#)
- [Ce que ce n'est pas](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)

4. Chapitre 4 - Variables CSS : une couleur, un endroit

- [Declarer et utiliser](#)
- [Pourquoi c'est genial sur une landing](#)
- [Valeur de secours](#)
- [Changer localement](#)
- [Noms qui aident](#)

- [Variables et formulaires](#)
- [Ce que ce n'est pas](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)

5. [Chapitre 5 - CSS Grid : lignes et colonnes](#)

- [Allumer la grille](#)
- [Colonnes plus expressives](#)
- [minmax et auto-fit](#)
- [Lignes et hauteur](#)
- [Gap, le petit heros](#)
- [Placement simple](#)
- [Alignement dans les cellules](#)
- [Ce que ce n'est pas](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)

6. [Chapitre 6 - Mise en page avec Grid : header, contenu, aside](#)

- [Le dessin de zones](#)
- [Brancher le HTML](#)
- [Variante avec menu lateral](#)
- [Empiler sur petit ecran](#)
- [Contenu dans les zones](#)
- [Header interne en Flex](#)
- [Zones nommees et debug](#)
- [Ce que ce n'est pas](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)

7. [Chapitre 7 - Flex vs Grid : quand choisir quoi](#)

- [Choisis Flex quand...](#)
- [Choisis Grid quand...](#)
- [Les deux ensemble](#)
- [Cas limites utiles](#)
- [Signes que tu as choisi le mauvais outil](#)
- [Mini decision rapide](#)
- [Ce que ce n'est pas](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)

8. [Chapitre 8 - Images modernes : taille, cadrage, srcset](#)

- [Ne jamais déborder : max-width](#)
- [Object-fit : cadrer sans déformer](#)
- [Le HTML reste responsable du fond](#)
- [Idée srcset, en simple](#)
- [Formats et poids](#)
- [Image dans une grille](#)
- [Ce que ce n'est pas](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)

9. [Chapitre 9 - Styler un formulaire proprement](#)

- [HTML d'abord, toujours](#)
- [Une colonne claire](#)
- [Focus visible](#)
- [Bouton cohérent](#)
- [Deux champs côte à côte \(desktop\).](#)
- [Etats d'erreur \(léger\).](#)
- [Case à cocher et radio](#)
- [Ce que ce n'est pas](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)

10. [Chapitre 10 - Transitions légères : du mouvement utile](#)

- [Ce que ce n'est pas](#)
- [Etat de base, puis hover](#)
- [Que transitionner ?](#)
- [Durées et courbes](#)
- [Liens, focus, et accessibilité du mouvement](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)

11. [Chapitre 11 - Accessibilité suite : focus, contrastes, labels](#)

- [Focus visible : savoir où on est](#)
- [Contrastes : lisible, pas juste joli](#)
- [Labels : dire le nom des champs](#)
- [Ordre de tabulation naturel](#)
- [Images et boutons](#)
- [Landing : checklist express](#)
- [Ce que ce n'est pas](#)
- [Petite histoire](#)

- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)

12. [Chapitre 12 - Mini-projet : page d'accueil responsive \(Grid + variables\)](#)

- [Brief](#)
- [Etape 1 - HTML semantique](#)
- [Etape 2 - Variables](#)
- [Etape 3 - Coquille et header](#)
- [Etape 4 - Hero](#)
- [Etape 5 - Grille de cartes](#)
- [Etape 6 a 8 - Formulaire, transitions, a11y](#)
- [Criteres de reussite](#)
- [Ce que ce n'est pas](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)

13. [Chapitre 13 - A retenir](#)

- [Structure et cascade](#)
- [Variables, Grid, Flex](#)
- [Images, formulaires, mouvement, accessibilite](#)
- [Ce que ce n'est pas](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)

14. [Chapitre 14 - Atelier Grid : blog en zones](#)

- [Ce que ce n'est pas](#)
- [Brief](#)
- [Etapas](#)
- [Amorce CSS](#)
- [Contenu minimum et criteres](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [Bonus](#)
- [A toi](#)

15. [Chapitre 15 - Atelier variables : une marque en `:root`](#)

- [Ce que ce n'est pas](#)
- [Brief](#)
- [Etapas](#)
- [Amorce palette + bascule](#)
- [Pastilles et criteres](#)

- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [Bonus](#)
- [A toi](#)

16. [Chapitre 16 - Atelier page : assembler une mini landing](#)

- [Ce que ce n'est pas](#)
- [Brief obligatoire](#)
- [Etapas](#)
- [Amorce hero + tarifs](#)
- [Contenu minimum et criteres](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [Bonus](#)
- [A toi](#)

17. [Chapitre 17 - Dark mode avec variables](#)

- [Ce que ce n'est pas](#)
- [Preference systeme](#)
- [Bascule manuelle, ombres, formulaires](#)
- [Contrastes, pas ambiance](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)

18. [Chapitre 18 - Perf CSS legeres : aller vite sans obsession](#)

- [Ce que ce n'est pas](#)
- [Ordre d'attaque](#)
- [Hygiene CSS et selecteurs](#)
- [Images, fonts, stabilite](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)

19. [Chapitre 19 - Debug CSS : outils, outline, hypotheses](#)

- [Ce que ce n'est pas](#)
- [Methode en cinq gestes](#)
- [L'inspecteur et l'outline](#)
- [Checklist des pieges frequents](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)

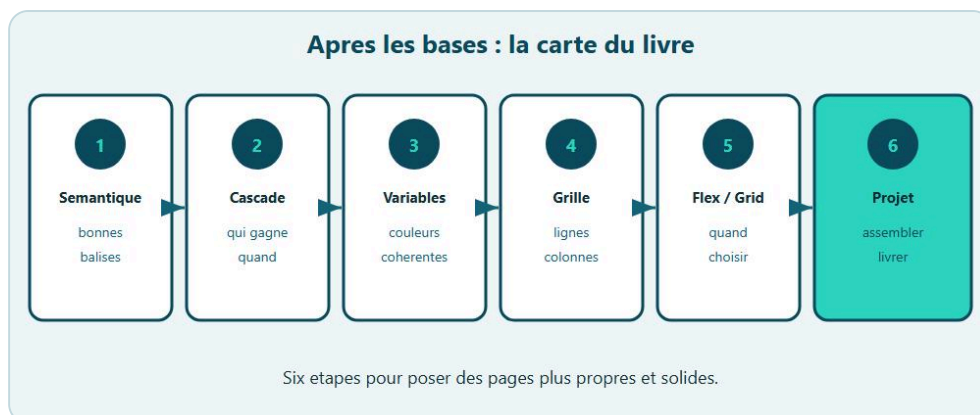
20. Quiz final

- [Questions](#)
- [Corriges](#)
- [Petite histoire](#)
- [Mini debrief](#)
- [En vrai](#)
- [A toi](#)

21. Bravo

- [Ce que tu sais faire maintenant](#)
- [Petite mission \(si tu veux\)](#)
- [Ce que ce n'est pas](#)
- [Petite scene DanielCraft](#)
- [En vrai](#)
- [La suite, quand tu seras chaud](#)
- [A toi](#)

Chapitre 1 - Rappel rapide et carte du livre



Après les bases : semantique, cascade, variables, Grid, a11y.

Tu as déjà fait le premier livre HTML et CSS. Tu sais structurer une page, écrire des balises, relier un fichier CSS, jouer avec les couleurs et les polices. Tu as vu les boîtes (margin, padding, border), **Flexbox** pour ranger les blocs, et une intro au responsive. Un peu d'accessibilité aussi. Ici, on ne recommence pas à zéro. Pas de cours "c'est quoi une marge" pendant trois pages. Si Flexbox te semble encore flou, relis le premier livre. Ce tome-ci monte d'un cran : pages plus propres, plus solides, plus jolies sans tricher.

Chez DanielCraft, on aime une image simple. Les bases, c'était poser les murs et peindre une pièce. La suite, c'est poser un vrai plan d'étage, choisir une palette qui tient partout, faire bouger un bouton sans sursaut, et penser à ceux qui naviguent autrement que toi. Lea, freelance web, s'en sert pour livrer des landings clients sans recommencer le CSS à chaque couleur. Max, artisan, veut juste une page devis claire sur téléphone. Sam, enseignant, veut montrer à ses élèves que "suite" ne veut pas dire "compliqué" - ça veut dire "mieux range".

Ce que tu gardes en tête

Tu sais écrire un HTML clair avec **h1**, **p**, **a**, **img**. Tu sais cibler avec des classes. Tu sais aligner avec Flexbox. Tu as déjà ouvert la page sur téléphone (ou réduit la fenêtre). C'est assez pour avancer. On ne revient pas sur "comment faire un lien" ni sur "display: flex" de A à Z. On s'en sert, oui. On ne le reexplique pas comme à un débutant total. Pareil pour margin et padding : tu les connais, on les utilise sans leçon débutant.

Ce que ce n'est pas

Ce n'est pas un second tome "bases bis". Ce n'est pas non plus un catalogue de frameworks. Pas de Bootstrap obligatoire, pas de Tailwind impose. Ici, on reste en HTML et CSS "nus", clairs, dans le navigateur. Et ce n'est surtout pas "je saute au mini-projet parce que je connais déjà les mots". Les mots aident. Les pages livrées prouvent.

La carte du livre

Imagine une page produit, un petit blog, ou une landing pour un atelier. Pour que ça tienne vraiment, il te manque encore quelques pièces. Voici la carte, dans l'ordre où on va les poser.

D'abord le HTML **semantique** : dire au navigateur "ca, c'est le menu", "ca, c'est l'article", "ca, c'est le pied de page". Ensuite la **cascade** et la specificite : comprendre pourquoi une regle gagne sur une autre, sans paniquer. Puis les **variables CSS** : une couleur declaree une fois, reutilisee partout.

Grid arrive ensuite : poser la page en lignes et colonnes, pas seulement en rangee Flexbox. On fera les bases, puis une vraie mise en page (header, contenu, aside). Juste apres, Flex vs Grid : savoir choisir l'outil, pas empiler les deux au hasard.

Les images modernes : taille, **object-fit**, une idee simple de **srcset**. Les formulaires styles proprement. Des transitions legeres (hover doux, pas de cirque). L'accessibilite suite : focus visible, contrastes, labels. Un mini-projet qui assemble grid et variables. Un recap. Trois ateliers pour faire, pas seulement lire. Le mode sombre avec les variables. Un peu de perf CSS. Du debug (outils, outline, hypotheses). Un quiz. Et un bravo final.

Un fil rouge

Garde un fil rouge des le debut : une page produit, un blog, ou une landing atelier. Les exemples du livre colleront mieux si tu as un vrai but sous la main. On va souvent parler de ces trois exemples. Une page produit (carte, prix, bouton). Un blog (en-tete, articles, pied). Une landing simple (hero, sections, appel a l'action). Ces exemples reviennent, pour que tu sentes le progres. Tu verras le meme geste sous plusieurs angles : structurer, thematiser, poser la grille, soigner le detail, verifier que ca reste utilisable au clavier.

Lea travaille surtout landings et cartes boutique. Max pense "page artisan + devis". Sam pense "demo en classe qui tient sur un telephone eleve". Trois metiers, meme progression.

♦ ASTUCE

Si tu as deja une page du premier livre, garde-la ouverte a cote : elle servira de terrain d'essai pour chaque chapitre.

Ce dont tu as besoin

Un editeur (VS Code, Cursor, ou autre). Un navigateur moderne. Les outils developpeur (F12). Pas de framework. Si tu as deja une page du premier livre, garde-la ouverte a cote.

Comment lire ce livre

Lis dans l'ordre au debut. Semantique, cascade, variables, puis Grid. Flex vs Grid apres. Images, formulaires, transitions, accessibilite avant le mini-projet. Les ateliers sont la pour faire. Dark mode, perf et debug solidifient. Le quiz verifie. Si tu connais deja un sujet, lis quand meme le chapitre vite : le ton et les exemples servent de rappel. A chaque fin, un "A toi" : fais-le.

Petite histoire

Lea a ouvert ce second tome en pensant "je connais deja CSS". Puis elle a retombe sur une landing client avec trois verts differents, un menu en **div**, et zero focus clavier. En suivant la carte - semantique, variables, Grid, a11y - elle a corrige en une soiree ce qu'elle trainait depuis des semaines. Max, lui, avait saute directement au "faire joli". Son neveu lui a dit de commencer par le plan. Sam a affiche la carte du livre au tableau : "on ne court pas, on assemble".

⚠ ATTENTION

Croire que "je connais les bases" = "je sais faire une vraie page pro" est le piège classique. Les bases sont le moteur. La suite, c'est le plan, la cohérence, le confort, et le respect des visiteurs.

Erreur classique

Sans sémantique, ta page est un tas de `div`. Sans variables, changer de couleur devient un cauchemar. Sans Grid (ou un plan clair), le layout casse dès que la fenêtre change. Sans accessibilité, tu excludes du monde sans le vouloir. Autre piège : sauter directement au mini-projet. Une page "jolie" qui casse au Tab ou qui mélange trois verts différents n'est pas finie.

En vrai

Ouvre une page que tu as déjà codée (page perso, carte produit...). Note ce qui manque : un vrai `header` / `main` ? une couleur copiée quinze fois ? un layout qui casse en large ? un bouton sans état focus ? un formulaire moche ? Ce livre répond à ça.

Si tu n'as aucune page sous la main, note trois pages web que tu aimes (boutique, blog, landing) et observe : où est le menu, où est le contenu, comment les cartes s'alignent, si le bouton change doucement au survol.

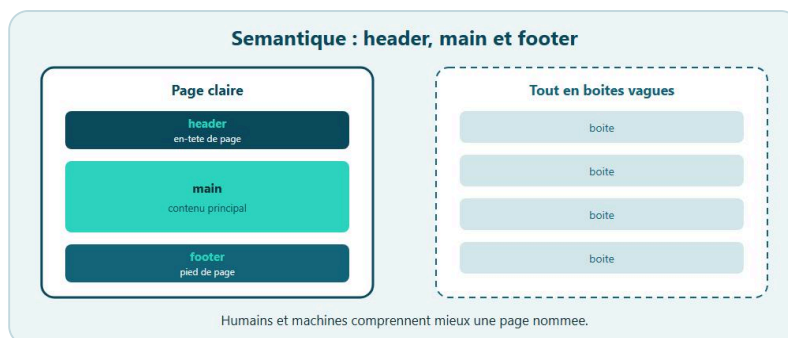
A toi

Écris en trois phrases ce que tu veux construire à la fin. Pas un réseau social. Quelque chose de petit : "une page produit propre", "un blog à deux articles", "une landing pour mon atelier". Garde ce but. On y reviendra au mini-projet et aux ateliers.

★ A RETENIR

Bases = moteur. Suite = plan + cohérence + confort. Lis dans l'ordre, fais les "A toi", vise une petite page réelle.

Chapitre 2 - HTML sémantique : donner un sens à la page



Nommer les pièces : header, main, nav, article, footer.

Tu peux construire toute une page avec des `div`. Ça marche. Le navigateur affiche. Mais pour un humain qui lit le code, pour un lecteur d'écran, pour un moteur de recherche, une pile de `div` anonymes, c'est un mur sans portes ni pièces nommées. Le HTML **sémantique**, c'est nommer les pièces. `header`, `nav`, `main`, `article`, `section`, `footer` (et d'autres). Tu dis ce que c'est, pas seulement "un bloc".

Chez DanielCraft, on traite ça comme le plan d'une boutique : vitrine, rayon, caisse, sortie. Si tout s'appelle "pièce", tu te perds. Si chaque zone a un rôle, la visite devient claire. Lea ouvre un fichier client six mois plus tard et retrouve le menu sans jouer aux détectives. Max comprend mieux sa page quand "entête / contenu / pied" sont des balises, pas des classes mystérieuses. Sam force ses élèves à expliquer la page à voix haute avec les noms de balises : si ça sonne juste, la structure tient.

Pourquoi ça compte vraiment

Le navigateur n'a pas besoin de `header` pour afficher un bandeau. Une `div` avec une classe ferait l'affaire pour le CSS. Alors pourquoi s'embêter ? Parce que le sens voyage. Un lecteur d'écran peut proposer "aller au contenu principal" grâce à `main`. Les titres dans un `article` ont un contexte. Les moteurs comprennent mieux la structure. Toi, dans six mois, tu rouvres le fichier et tu sais où est le menu.

Sur une landing, le hero n'est pas "div 1". C'est souvent un `header` ou une `section` claire dans le `main`. Sur un blog, chaque billet est un `article`. Sur une page produit, la fiche peut être un `article` ou une `section` bien titrée. Le CSS viendra poser Grid ou Flex dessus. Le sens est là avant la peinture.

♦ ASTUCE

Avant de styler, lis ton HTML à voix haute : "en-tête, menu, contenu principal, article, pied". Si ça sonne comme un sommaire, tu es sur la bonne voie.

Les balises de structure que tu vas utiliser

`header` : en-tête d'une page ou d'une section (logo, titre, parfois le menu). Commence simple : un en-tête de page. `nav` : menus reels, pas chaque lien isolé. `main` : le contenu principal unique - en général un seul par page. `article` : un contenu qui tient tout seul (billet, fiche produit). `section` : regroupement thématique avec souvent un titre (Services, Contact...). `footer` : pied de page ou d'article. `aside` : contenu complémentaire à côté. Pas obligatoire partout.

Tu auras encore des `div` pour le layout neutre. Sémantique = nommer les zones importantes. Si tout est `section` sans réfléchir, tu n'as gagné que du bruit.

Une page blog, version sémantique

```
<body>
  <header>
    <p class="logo">Mon Blog</p>
    <nav aria-label="Principale">
      <a href="/">Accueil</a>
      <a href="/articles">Articles</a>
    </nav>
  </header>

  <main>
    <article>
      <h1>Titre du billet</h1>
      <p>Intro du texte...</p>
    </article>
  </main>

  <aside>
    <h2>À lire aussi</h2>
    <ul>
      <li><a href="#">Autre billet</a></li>
    </ul>
  </aside>

  <footer>
    <p>Contact - Mentions</p>
  </footer>
</body>
```

Tu vois déjà le plan. Le CSS viendra poser Grid ou Flex dessus. Le sens est là avant la peinture.

Landing simple

Sur une landing atelier, tu peux faire :

```
<header>
  <p class="marque">Atelier Ceramique</p>
  <nav>...</nav>
</header>
<main>
  <section class="hero">
    <h1>Apprends la terre en un week-end</h1>
    <p>Petit groupe, vrai four, vrai fun.</p>
    <a class="bouton" href="#inscription">Je m'inscris</a>
  </section>
  <section id="programme">
    <h2>Programme</h2>
    ...
  </section>
</main>
<footer>...</footer>
```

Le **h1** reste unique et fort. Les **h2** portent les sections. Tu ne sautes pas de **h1** à **h3** sans raison. Lea livre souvent ce squelette avant toute couleur. Max le reconnaît : "c'est comme afficher clairement l'entrée, le comptoir, la sortie".

Carte produit

Une grille de cartes peut être une liste d' `article` :

```
<main>
  <h1>Nos cafes</h1>
  <div class="grille">
    <article class="carte">
      <h2>Blend maison</h2>
      <p>Notes de cacao.</p>
      <p class="prix">9,90 €</p>
    </article>
  </div>
</main>
```

Le `div.grille` sert au layout CSS. Les `article` portent le sens de chaque produit. `div` = boîte neutre. Sémantique = boîte qui dit son rôle.

Ce que ce n'est pas

Ce n'est pas une religion ni "ça fait pro" donc j'enveloppe tout le body dans un `section` géant". Un nom sans rôle clair = bruit.

Petite histoire

Sam a demandé à un élève de décrire sa page sans regarder l'écran. L'élève a dit "plein de divs". Ils ont renommé : `header`, `main`, deux `article`, `footer`. Même CSS. L'oral est devenu clair en trente secondes. Lea a vécu le même moment avec un client qui voulait "juste changer le menu" : avec un vrai `nav`, la cible était évidente. Max a remplacé son bandeau `div.haut` par un `header` et a soudain retrouvé où coller le téléphone.

⚠ ATTENTION

Trois `main` sur la même page, ou un `header` qui avale tout le contenu principal : tu gagnes du jargon, tu perds le sens. Un `main`, un `h1` fort, des zones nommées avec intention.

Erreur classique

Mettre trois `main` sur la même page. Ou envelopper tout le body dans un seul `section` géant sans titre. Ou utiliser `header` uniquement parce que "ça fait pro", puis y coller le contenu principal entier. Autre piège : confondre `nav` et une liste de liens sociaux dans le pied. Un groupe de liens peut rester une liste dans le `footer` sans `nav`, ou avec un `nav` clairement libellé si c'est vraiment de la navigation.

En vrai

Prends une de tes pages. Remplace mentalement (puis dans le code) les gros `div` par `header`, `main`, `footer`. Ajoute `nav` autour du vrai menu. Si tu as un billet ou une fiche, tente `article`. Recharge. Visuellement, rien ne change si le CSS ciblait des classes. Structurellement, la page respire mieux.

Ouvre ensuite l'inspecteur : l'arbre DOM se lit comme un sommaire. C'est exactement ce que tu veux pour la suite (Grid, accessibilité, debug).

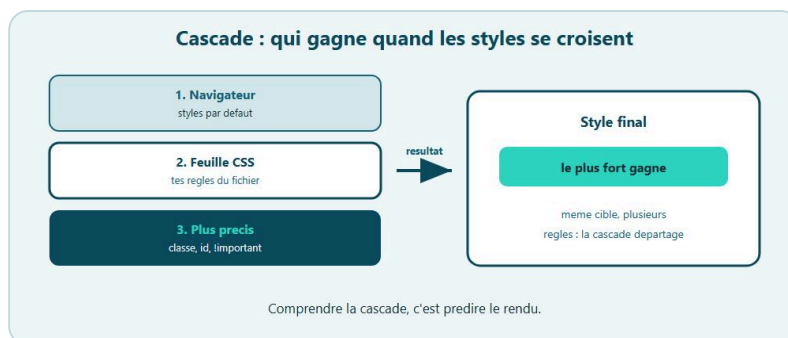
A toi

Recree une mini page "Produit" en HTML seul (presque pas de CSS). `header` avec nom de boutique + `nav`, `main` avec un `article` produit (titre, paragraphe, prix), `footer` avec une ligne de contact. Valide que tu as un seul `h1` et un seul `main`.

★ A RETENIR

Semantique = nommer les pieces. Un `main`, un `h1` fort. Les `div` restent pour le neutre. Le sens avant la peinture.

Chapitre 3 - Cascade et specificite : qui gagne ?



Cascade et specificite : qui gagne, sans paniquer.

Tu écris une règle. Tu en écris une autre. La couleur ne change pas. Ou elle change, mais pas celle que tu voulais. Bienvenue dans la **cascade**. Ce n'est pas un monstre. C'est l'ordre dans lequel le navigateur décide quelle règle CSS l'emporte. La **specificite**, c'est le "poids" d'un sélecteur. Ensemble, elles expliquent presque tous les "pourquoi ça marche pas".

Chez DanielCraft, on compare ça à des couches de peinture : la dernière couche visible gagne, sauf si une couche précédente était beaucoup plus "forte" (vernis special). Tu n'as pas besoin de jargon opaque. Tu as besoin d'instinct. Lea ouvre l'inspecteur avant de crier au bug. Max appelle ça "trouver qui commande le bouton". Sam note au tableau : ordre, poids, inline, **!important** - dans cet ordre de suspicion.

L'ordre compte

À poids égal, la règle la plus basse dans le fichier (la dernière lue) gagne en général.

```
.titre {
  color: blue;
}

.titre {
  color: green;
}
```

Le titre sera vert. Deuxième déclaration, même sélecteur : elle écrase la première. Si tu charges deux feuilles CSS, l'ordre des `<link>` compte aussi. La feuille liée en dernier peut écraser la précédente, à specificite égale.

Qu'est-ce qui pèse plus ?

Sans rentrer dans une formule magique à retenir par cœur, retiens cette échelle simple, du plus faible au plus fort. Un sélecteur de type (`p`, `h1`) pèse peu. Une **classe** (`.carte`, `.bouton`) pèse plus. Un identifiant (`#menu`) pèse encore plus. Les styles inline dans le HTML (`style="..."`) pèsent très fort (à éviter pour le travail sérieux). **!important** force encore plus fort (à utiliser presque jamais).

```
p {
  color: black;
}

.intro {
  color: #1a5f4a;
}

#accroche {
  color: red;
}
```

Sur `<p id="accroche" class="intro">`, le rouge de `#accroche` gagne. Pas parce qu'il est "mieux", parce qu'il pèse plus.

Classes plutôt que ID pour le style

Les ID sont utiles en HTML (cibles de liens, labels `for`). Pour le CSS du quotidien, préfère les classes. Pourquoi ? Parce qu'un ID trop présent dans le CSS rend les overrides douloureux. Tu te retrouves à monter en `!important` ou à empiler des sélecteurs monstrueux.

Sur une landing, `.hero-titre` se réutilise et se surcharge proprement. `#hero-titre` devient un piège des que tu veux une variante. Lea a appris ça en livrant une variante "promo" : avec des classes, dix minutes ; avec des ID, une soirée de bagarre.

♦ ASTUCE

Quand une couleur "refuse" de changer, F12 > Styles. Cherche la propriété barrée. Tu vois le gagnant. Ensuite tu corriges avec intention : ordre, poids, ou sélecteur plus clair - pas au hasard.

Sélecteurs composés

`.carte .prix` est plus spécifique qu'un simple `.prix`. Normal : tu as ciblé plus précisément.

```
.prix {
  color: #333;
}

.carte .prix {
  color: #1a5f4a;
}
```

Dans une carte produit, le prix prend le vert. Ailleurs, un `.prix` seul reste gris foncé. Attention à ne pas écrire des sélecteurs kilométriques du type `body div main section article div p span`. Plus c'est long, plus c'est fragile.

Héritage (rapide)

Certaines propriétés se transmettent aux enfants (couleur de texte, police...). D'autres non (margin, padding, border en général). Si un paragraphe "prend" la couleur du `body`, ce n'est pas toujours la cascade qui écrase : c'est l'**héritage**. Quand tu débogues : demande-toi "est-ce une règle qui s'applique à cet élément, ou une valeur héritée du parent ?"

Conflit classique sur une carte

```
a {
  color: blue;
}

.carte a {
  color: inherit;
}

.bouton {
  color: white;
  background: #1a5f4a;
}
```

Si ton bouton est un lien `<a class="bouton">`, tu peux avoir une bataille entre `a`, `.carte a` et `.bouton`. Regarde dans l'inspecteur quelle règle est barrée. C'est la meilleure leçon. Sam fait exactement cet exo en classe : les élèves voient le gagnant avant de "deviner".

!important, le extincteur

`!important` éteint l'incendie... et détruit souvent la maison ensuite. Une fois que tu en mets partout, plus rien n'est prévisible. Garde-le pour des cas rares (utile temporairement pour debug, ou contrainte très particulière). Le vrai fix, c'est : sélecteur un peu plus clair, ordre du fichier, moins d'ID.

Ce que ce n'est pas

Ce n'est pas une formule à recracher par cœur le jour du quiz. Ce n'est pas non plus "mettre un ID partout pour être sûr de gagner". Tu gagnes aujourd'hui, tu perds demain. Et ce n'est surtout pas modifier le HTML en inline `style=""` parce que "ça marche tout de suite". Ça marche, et ça pollue.

⚠ ATTENTION

`!important` et les styles inline calment la panique une minute, puis rendent la cascade illisible. Préfère classes claires, ordre du fichier, inspecteur.

Petite histoire

Lea avait un bouton CTA qui restait bleu "lien" malgré sa classe `.bouton`. L'inspecteur montrait `.carte a` plus fort. Elle a clarifié le sélecteur du bouton, retiré un `!important` temporaire, et respire. Max avait collé `style="color:red"` sur un titre "pour voir". Son neveu a refusé de toucher le fichier tant que l'inline était là. Sam appelle ça "la règle du vernis : si tout est vernis, plus rien ne se voit".

Erreur classique

Penser "je vais mettre un ID, comme ça je suis sûr que ça gagne". Ou empiler trois classes inventées (`.bloc.special.urgent`) au lieu de clarifier la structure. Autre piège : modifier le HTML en inline parce que ça marche tout de suite. Autre encore : paniquer et tout mettre en `!important`.

En vrai

Ouvre une page ou un style "refuse" de s'appliquer. F12, inspecte l'element, onglet Styles. Cherche la propriete barree. Remonte au selecteur gagnant. Demande-toi : ordre ? poids ? inline ? Une fois que tu vois le gagnant, tu corriges avec intention.

Fais le meme exercice sur une landing avec menu + bouton. Regarde quelle regle colore les liens du menu vs le bouton CTA.

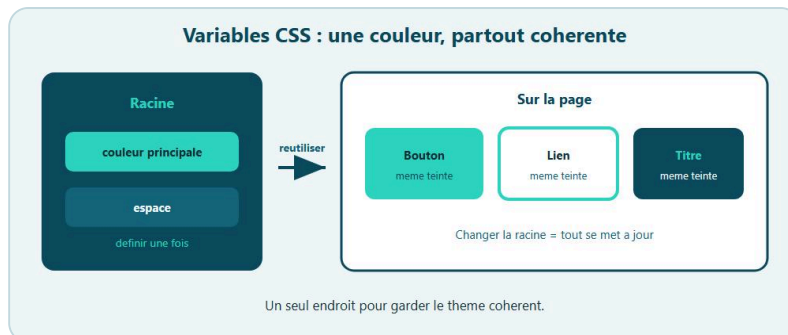
A toi

Cree une mini page avec un paragraphe `.intro` et un titre `#titre` (oui, un ID, pour l'exercice). Ecris volontairement des regles conflictuelles (type, classe, ID) sur la couleur. Note sur papier qui gagne avant d'ouvrir l'inspecteur. Verifie. Ensuite, retire l'ID du CSS et refais le style uniquement avec des classes. Garde la lecon.

★ A RETENIR

A poids egal, la derniere regle gagne. Classes > types. Evite ID et `!important` pour le style quotidien. L'inspecteur montre qui gagne.

Chapitre 4 - Variables CSS : une couleur, un endroit



Variables CSS : une couleur, un endroit.

Tu as deja change une couleur primaire dans dix classes differentes. Tu as oublie la onzieme. La page a l'air batarde. Ca arrive a tout le monde. Les **variables CSS** (custom properties) resolvent ca. Tu declares une valeur une fois, souvent dans `:root`, puis tu la reutilises partout avec `var(...)`.

`:root`, c'est la racine du document. En pratique : "mes reglages globaux pour toute la page". Chez DanielCraft, une palette claire, c'est une marque qui tient. Variables = palette centralisee. Lea refuse desormais de livrer une carte produit sans `:root`. Max comprend l'idee quand on lui dit "une peinture pour tout l'atelier, pas un pot par mur". Sam bascule le theme en cours devant la classe : les eleves voient le pouvoir d'une ligne.

Declarer et utiliser

```
:root {
  --couleur-principale: #1a5f4a;
  --couleur-fond: #f7f5f0;
  --couleur-texte: #222;
  --espace: 1rem;
}

body {
  background: var(--couleur-fond);
  color: var(--couleur-texte);
}

a {
  color: var(--couleur-principale);
}

.bouton {
  background: var(--couleur-principale);
  color: #fff;
  padding: var(--espace) calc(var(--espace) * 1.5);
}
```

Le double tiret `--` marque le nom de la variable. Tu choisis des noms clairs : `--couleur-principale` plutot que `--c1`. Une **palette** digne de ce nom, c'est ca : des roles nommes, pas des codes magiques eparpilles. Nomme par role (`--couleur-principale`, `--fond`, `--texte`) plutot que par teinte (`--vert`, `--beige`). Les roles survivent aux restyles. Les teintes se periment.

Pourquoi c'est genial sur une landing

Imagine une landing avec bouton, liens, bordures d'accent, titre surligne. Si demain tu passes du vert au bleu marine, tu changes une ligne dans `:root`. Toute la page suit. C'est ca, une vraie palette.

Tu peux aussi stocker des tailles, des rayons, des ombres legeres :

```
:root {
  --rayon: 8px;
  --ombre: 0 4px 12px rgba(0, 0, 0, 0.08);
  --largeur-max: 1100px;
}

.carte {
  border-radius: var(--rayon);
  box-shadow: var(--ombre);
}

.wrap {
  max-width: var(--largeur-max);
  margin-inline: auto;
}
```

Sur un blog, le meme `--espace` rythme le padding des articles et le gap de la grille. Moins de magie, plus de coherence. Lea touche trois variables et la marque client bascule. Max change `--couleur-principale` et son bouton devis suit.

★ A RETENIR

`:root + var(...)` = palette centralisee. Noms de role. Une ligne change, toute la marque suit.

Valeur de secours

`var(--couleur-principale, #1a5f4a)` utilise la variable si elle existe, sinon le second argument. Utile quand tu experimentes ou quand une variable pourrait manquer.

```
.badge {
  background: var(--couleur-accent, #c45c26);
}
```

Changer localement

Une variable n'est pas obligatoirement globale. Tu peux la redefinir dans un bloc :

```
.carte-promo {
  --couleur-principale: #8b1e3f;
}

.carte-promo .bouton {
  background: var(--couleur-principale);
}
```

La carte promo a son accent. Le reste du site garde le vert. Les enfants de `.carte-promo` voient la nouvelle valeur. C'est puissant pour des themes de section sans dupliquer tout le CSS. Sam adore cet exemple : "meme bouton, autre contexte".

Noms qui aident

Mieux vaut `--couleur-principale` et `--couleur-fond` que `--vert` et `--beige` si tu comptes changer de teinte. Pour une boutique : `--prix`, `--dispo`, `--rupture` peuvent aussi etre des variables de couleur sémantiques.

Variables et formulaires

```
:root {
  --champ-bordure: #ccc;
  --champ-focus: #1a5f4a;
}

input {
  border: 1px solid var(--champ-bordure);
}

input:focus {
  border-color: var(--champ-focus);
  outline-color: var(--champ-focus);
}
```

Tu prepares deja le terrain pour le chapitre formulaires et pour le `dark mode` plus tard. Meme reflexe, autre palette.

Ce que ce n'est pas

Ce n'est pas declarer vingt variables "au cas ou" sans les utiliser. Ce n'est pas non plus copier la meme valeur en dur dans une classe "parce que c'est plus rapide" - tu reviens au probleme initial. Et ce n'est pas encore le dark mode systeme (chapitre 17), meme si tu t'en approches.

⚠ ATTENTION

Oublier qu'un nom est sensible a la casse et aux tirets. `--Couleur` n'est pas `--couleur`. Et un hex "temporaire" dans `.bouton` reste presque toujours.

Petite histoire

Lea a livre une page avec theme clair uniquement. Le client a demande "version nuit". Elle a ajoute une seconde palette en vingt minutes parce que tout etait deja en variables. La fois d'avant, sans variables, ca avait pris trois heures et casse deux contrastes. Max a vu la demo et a demande la meme chose pour sa page devis. Sam a note le cas pour le cours suivant.

Erreur classique

Declarer vingt variables inutiles. Ou garder `#1a5f4a` dans `.bouton` "temporairement". Autre piege : oublier la casse. Autre encore : croire que les variables "ralentissent" le site - non : elles accelerent ta maintenance.

En vrai

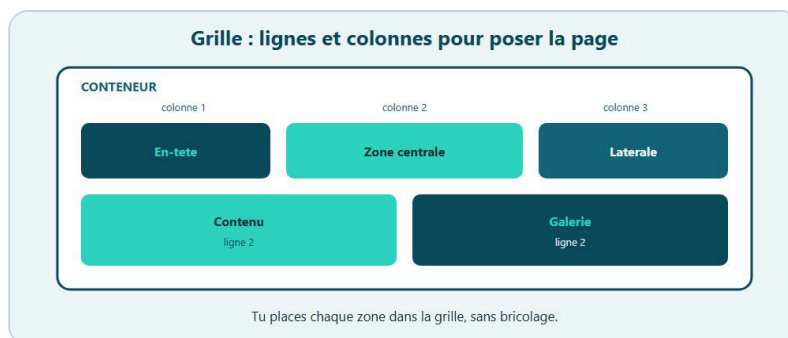
Ouvre une page existante. Liste les couleurs qui reviennent (boutons, liens, titres). Cree un `:root` avec trois a six variables. Remplace les valeurs en dur. Change une variable. Souris si toute la page suit.

Fais la meme chose pour `--espace` sur une grille de cartes produit : padding de carte et `gap` du parent.

A toi

Cree une mini landing : titre, paragraphe, bouton, une carte. Tout le theme passe par `:root` (`--couleur-principale`, `--fond`, `--texte`, `--rayon`, `--espace`). Change uniquement `:root` pour obtenir une variante "soir" (fond plus sombre, texte clair). Ne touche pas aux classes. Si ca marche, tu as compris les variables.

Chapitre 5 - CSS Grid : lignes et colonnes



Grid : lignes, colonnes, gap.

Flexbox, tu le connais : super pour une rangée, un menu, aligner dans un sens. **Grid**, c'est autre chose. Tu poses un vrai damier. Lignes et colonnes. Idéal pour une page, une galerie, un tableau de cartes produit. Tu n'abandonnes pas Flexbox. Tu ajoutes un outil. Souvent : Grid pour la structure de page, Flexbox pour les petits alignements à l'intérieur d'une cellule.

Chez DanielCraft, Grid, c'est le plan au sol. Flex, c'est ranger les chaises dans une pièce. Lea pose d'abord la grille de cartes boutique, ensuite le Flex du footer de carte (prix + bouton). Max voit "rayons dans l'atelier". Sam dessine des colonnes au tableau avant d'ouvrir l'éditeur.

Allumer la grille

```
.grille {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;
  gap: 1rem;
}
```

1fr veut dire "une part de l'espace libre". Trois colonnes égales, avec un espace **gap** entre les cases. Pas besoin de margin bridouilles entre chaque carte.

```
<div class="grille">
  <article class="carte">Produit A</article>
  <article class="carte">Produit B</article>
  <article class="carte">Produit C</article>
</div>
```

Sur une page boutique, ces trois cartes s'alignent proprement. Ajoute un quatrième produit : il passe à la ligne suivante, toujours dans la grille. Le **gap** est le petit héros : il espace sans casser les bords externes. Commence par colonnes + **gap**. Les lignes se créent souvent toutes seules. N'empile pas dix placements manuels tant que le flux HTML suffit.

Colonnes plus expressives

Tu peux mélanger unités :

```
.page {
  display: grid;
  grid-template-columns: 200px 1fr;
  gap: 2rem;
}
```

Colonne fixe a gauche (menu), le reste a droite (contenu). Ou l'inverse pour une landing avec une grosse zone texte et une colonne étroite.

`repeat` evite de se repeter :

```
.galerie {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 1rem;
}
```

minmax et auto-fit

Pour une grille de cartes qui s'adapte sans media query compliquee :

```
.grille {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
  gap: 1rem;
}
```

L'idée : chaque colonne fait au moins 220px, et on en met autant que ca rentre. Sur telephone, une colonne. Sur grand ecran, plusieurs. Tu peaufineras au chapitre mise en page et au mini-projet. Lea adore `auto-fit` + `minmax` pour les galeries produit : moins de breakpoints, plus de fluidite.

✦ ASTUCE

Relie `gap` a une variable `--gap` : tu retouches le rythme de toute la grille en une ligne, et tu peux le reduire sous media query.

Lignes et hauteur

Souvent, les lignes se creent toutes seules selon le contenu. Tu peux aussi les guider :

```
.hero-grille {
  display: grid;
  grid-template-columns: 1fr 1fr;
  grid-template-rows: auto auto;
  gap: 1.5rem;
}
```

Pour commencer, concentre-toi sur les colonnes et le `gap`. Les lignes suivront.

Gap, le petit heros

`gap` (et `row-gap` / `column-gap` si besoin) espace les cellules sans casser les bords externes. Sur un blog en grille de billets, c'est net, previsible, facile a retoucher.

```
:root {
  --gap: 1.25rem;
}

.grille {
  gap: var(--gap);
}
```

Placement simple

Sans dessiner toute la page, tu peux deja etendre un element :

```
.carte-large {
  grid-column: 1 / -1;
}
```

Cette carte occupe toute la largeur de la grille (de la premiere a la derniere ligne de colonnes). Utile pour un article eingle au-dessus d'une grille de billets.

```
.promo {
  grid-column: 2;
  grid-row: 1;
}
```

Reste lisible. N'en abuse pas au debut : une grille bien remplie dans l'ordre du HTML suffit souvent. Sam rappelle : "placement manuel = epice, pas plat principal".

Alignement dans les cellules

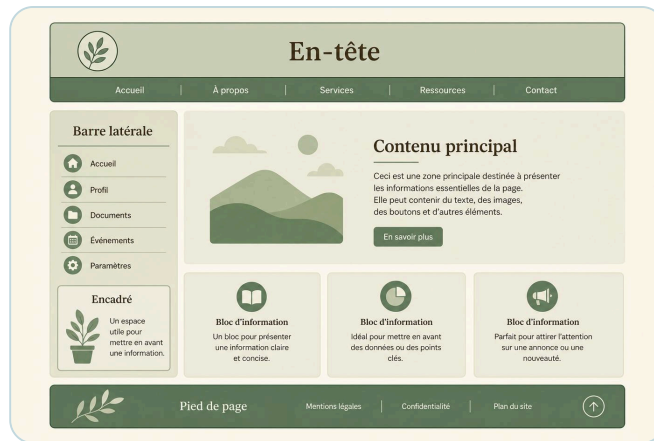
`justify-items` et `align-items` influencent le contenu dans chaque case. `justify-content` / `align-content` influencent la grille dans son conteneur quand il reste de l'espace. Pour une galerie de cartes, commence par laisser les items s'etirer (`stretch` par default) et controle le padding dans `.carte`.

Ce que ce n'est pas

Ce n'est pas "Flexbox est mort". Ce n'est pas non plus mettre `display: grid` et s'attendre a ce que Flex "merge" magiquement. Non : tu choisiss l'outil du conteneur. Les enfants deviennent des items de grille. Et ce n'est pas dix colonnes fixes en pixels sur mobile.

⚠ ATTENTION

Trop de colonnes fixes en pixels + `gap` enorme sur petit ecran etouffe le contenu. Teste en fenetre etroite. Passe `--gap` en plus petit sous media query si besoin.



Exemple : une page decoupee en zones claires.

Petite histoire

Lea a remplacé un Flex + largeurs en % bancal par `repeat(auto-fit, minmax(220px, 1fr))`. Les cartes boutique se sont alignées sans media query. Max a vu la demo et a dit "c'est comme des etageres qui s'adaptent a la piece". Sam a ajouté une carte "A la une" avec `grid-column: 1 / -1` : les élèves ont compris le placement en une minute.

Erreur classique

Mettre `display: grid` et esperer un hybride Flex. Autre piege : trop de colonnes fixes sur mobile. Autre : placement manuel partout alors que l'ordre HTML suffisait. Autre encore : oublier `gap` et compenser avec des margins sur chaque carte.

En vrai

Prends trois cartes produit (meme fausses). Mets-les dans une grille a deux ou trois colonnes avec `gap`. Redimensionne la fenetre. Observe comment les items se placent. Compare mentalement a un Flexbox avec `flex-wrap` : Grid pense deja en colonnes declarees.

Ajoute une quatrieme carte "A la une" avec `grid-column: 1 / -1` au-dessus. Tu sens deja le pouvoir du placement.

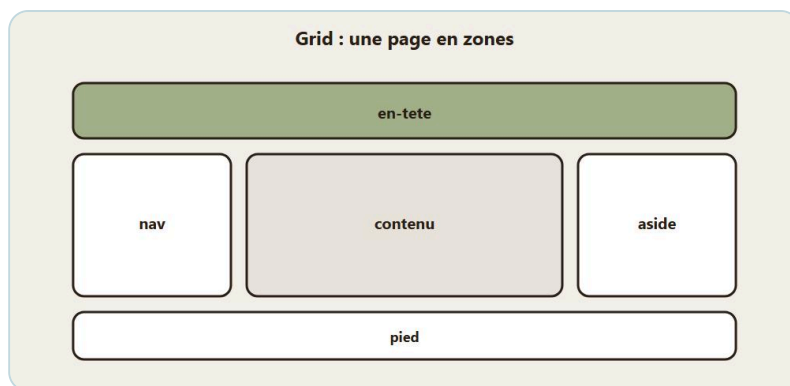
A toi

Cree une mini page "Ateliers" avec un titre et une grille de quatre cartes (titre + une phrase). `grid-template-columns: 1fr 1fr`, `gap: 1rem`. Sur feuille externe. Variables pour couleurs et gap. Pas besoin d'images. Le placement compte.

★ A RETENIR

Grid = lignes + colonnes. `fr`, `gap`, `repeat`, `minmax` / `auto-fit`. Flex reste pour une dimension. Un mode par conteneur.

Chapitre 6 - Mise en page avec Grid : header, contenu, aside



Header, nav, contenu, aside, pied : la grille de page.

Les bases de Grid, tu les as. Maintenant on pose une vraie page : en-tete, contenu, colonne laterale, pied. C'est le squelette d'un blog, d'une doc simple, d'une page "ressources" avec menu. L'outil star ici : **grid-template-areas**. Tu dessines la page avec des mots. Ensuite chaque zone recoit son nom. C'est lisible. C'est modifiable. C'est parfait pour un PDF de formation comme chez DanielCraft : tu vois le plan avant le detail.

Lea fait ce genre de squelette avant chaque refonte blog client : zones d'abord, deco ensuite. Max n'a pas de blog, mais il comprend "entete / contenu / aside / pied" comme les pieces d'un atelier. Sam donne exactement ce brief a ses eleves : un plan lisible avant le pixel perfect.

Le dessin de zones

```
.layout {
  display: grid;
  grid-template-columns: 2fr 1fr;
  grid-template-areas:
    "entete entete"
    "contenu aside"
    "pied pied";
  gap: 1.5rem;
  max-width: 1100px;
  margin-inline: auto;
  padding: 1rem;
}
```

Chaque ligne du dessin a le meme nombre de cellules. Ici : deux colonnes. L'entete prend les deux. Le contenu et l'aside se partagent la rangee du milieu. Le pied prend les deux. Si tu ecris "contenu" seul alors que tu as deux colonnes, le navigateur rale ou se comporte bizarrement. Compte les mots.

Brancher le HTML

```
<div class="layout">
  <header class="entete">...</header>
  <main class="contenu">...</main>
  <aside class="aside">...</aside>
  <footer class="pied">...</footer>
</div>
```

```
.entete { grid-area: entete; }
.contenu { grid-area: contenu; }
.aside { grid-area: aside; }
.pied { grid-area: pied; }
```

Le nom dans `grid-area` doit matcher le mot du dessin. Si tu écris `grid-area: header` mais que le dessin dit `entete`, ça ne se place pas. Une faute de typo et toute la grille "glisse".

♦ ASTUCE

Colorie temporairement chaque zone (fonds pastels différents) pour voir qui est où. Projecteur de debug, pas deco finale. Tu enlèves après.

Variante avec menu lateral

Sur grand écran, un blog peut aussi avoir :

```
.layout {
  grid-template-columns: 180px 1fr 240px;
  grid-template-areas:
    "entete entete entete"
    "nav contenu aside"
    "pied pied pied";
}
```

Trois colonnes. Le `nav` à gauche, le contenu au centre, l'aside à droite. Sur une landing marketing, tu n'as peut-être pas besoin d'aside : simplifie. Lea coupe l'aside dès qu'il n'apporte rien. Max préfère souvent deux zones : contenu + pied.

Empiler sur petit écran

```
@media (max-width: 700px) {
  .layout {
    grid-template-columns: 1fr;
    grid-template-areas:
      "entete"
      "contenu"
      "aside"
      "pied";
  }
}
```

Une colonne. Ordre lisible : en-tête, contenu d'abord (priorité), puis aside, puis pied. Si tu as un `nav` séparé, décide s'il passe juste sous l'entête ou après le contenu. Pour un blog, menu haut puis articles reste naturel. L'ordre HTML guide aussi le Tab : contenu avant aside, pas l'inverse "pour aider le CSS desktop".

Contenu dans les zones

Dans `main.contenu`, tu peux encore avoir une grille interne pour les articles :

```
.liste-articles {
  display: grid;
  gap: 1rem;
}
```

Ou deux colonnes d'articles sur tablette. Grid imbriquée : normale et saine. La page a un Grid de structure ; une section a un Grid de cartes. Dans l'aside : titre + liste de liens. Pas besoin d'en faire une usine.

Header interne en Flex

Souvent, le `header` en zone Grid utilise **Flexbox** à l'intérieur : logo à gauche, liens à droite.

```
.entete {
  display: flex;
  justify-content: space-between;
  align-items: center;
  gap: 1rem;
  flex-wrap: wrap;
}
```

Tu vois la complémentarité : Grid pose les pièces de la page, Flex range le contenu d'une pièce. C'est le réflexe DanielCraft pour presque toute landing propre.

Zones nommées et debug

Si tu te perds, colore temporairement :

```
.entete { background: #e8f5ef; }
.contenu { background: #fff8e8; }
.aside { background: #eef0ff; }
.pied { background: #f3f3f3; }
```

Tu vois immédiatement qui est où. Enlève les fonds après. Active aussi le mode grille de l'inspecteur sur `.layout`.

Ce que ce n'est pas

Ce n'est pas un concours de CSS créatif. Ce n'est pas `position: absolute` pour "faire tenir" le blog. Et ce n'est pas oublier le responsive : un aside étroit en pixels fixes à côté d'un contenu sur téléphone, ça étouffe. Repasse en une colonne.

⚠ ATTENTION

Oublier de redéfinir `grid-template-areas` dans le media query te laisse avec deux colonnes écrasées sur téléphone. Chaque breakpoint a son dessin.

Petite histoire

Sam a vu un élève mettre l'aside avant le `main` dans le HTML "pour aider le CSS desktop". Sur mobile, au Tab, l'aside arrivait avant les articles. Le correctif : ordre de lecture contenu puis aside, et laisser Grid placer. Lea a eu le même réflexe sur un projet client. Max, en voyant la demo, a dit : "c'est comme ranger l'atelier pour que le client trouve le devis avant les factures fournisseurs". Exactement.

Erreur classique

Des lignes du dessin avec un nombre de cellules different. Oublier `grid-area` sur un enfant. Oublier le responsive. Copier un layout trouve sur le web sans comprendre les noms de zones. Typo dans un nom de zone.

En vrai

Construis le squelette d'un blog : deux articles dans le `main`, trois liens dans l' `aside`, un vrai `header` / `footer`. Areas sur desktop, pile sur mobile. Redimensionne lentement. Note a quelle largeur ca devient serre - c'est ton breakpoint.

Compare avec une ancienne page ou tu avais tout fait en Flex + largeurs en %. Le dessin areas se lit souvent plus clairement.

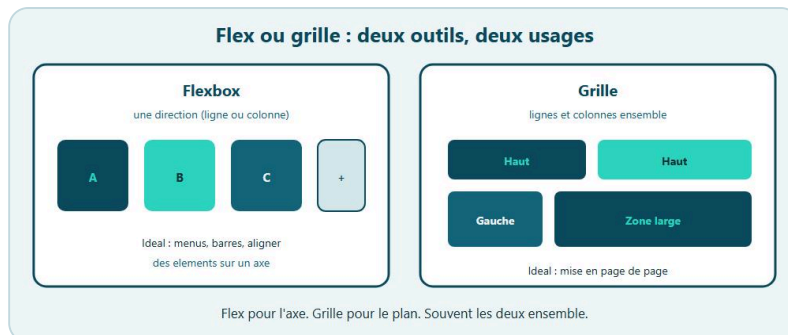
A toi

Livre une page "Blog atelier" complete (HTML + CSS). Layout areas comme ci-dessus. Variables pour gap et couleurs. Sur `max-width: 700px`, une colonne. Critere : sans CSS, le HTML reste dans un ordre logique (entete, articles, aside, pied).

★ A RETENIR

`grid-template-areas` = plan en mots. Meme nombre de cellules par ligne. Grid dehors, Flex dedans. Une colonne sur mobile.

Chapitre 7 - Flex vs Grid : quand choisir quoi



Flex ou Grid : choisir le bon outil.

Tu as deux outils puissants. La question n'est plus "lequel est le meilleur ?" mais "lequel pour ce job ?".

Flexbox brille dans une dimension : une rangée ou une colonne. Aligner, distribuer l'espace, centrer un bouton, faire un menu, coller une icône à un label. **Grid** brille en deux dimensions : lignes et colonnes en même temps. Page entière, galerie, tableau de cartes, zones nommées.

Chez DanielCraft, on dit souvent : Grid pour le plan de la page, Flex pour le mobilier dans chaque pièce. Ce n'est pas une loi absolue. C'est un excellent réflexe de départ. Lea le note en commentaire CSS au-dessus de chaque conteneur. Max s'en fiche des noms tant que le devis tient sur téléphone - mais le réflexe l'aide quand son neveu explique. Sam fait voter la classe avant de révéler l'outil : "rangée de liens ou damier ?"

Choisis Flex quand...

Tu ranges des éléments sur une seule ligne (ou une seule colonne) et tu veux contrôler l'espace entre eux. Exemples concrets : un menu horizontal (logo + liens), un bandeau (titre à gauche, bouton à droite), une carte produit (image au-dessus, puis en bas une rangée prix + bouton en Flex). Centrer un bloc dans un hero (souvent Flex sur le conteneur, ou Grid avec place-items - les deux marchent).

```
.menu {
  display: flex;
  gap: 1rem;
  align-items: center;
  flex-wrap: wrap;
}
```

Simple, direct, parfait. Une **dimension**, un outil clair.

Choisis Grid quand...

Tu penses déjà "colonnes" et "lignes". Tu veux qu'un élément occupe toute la largeur pendant que d'autres se partagent une rangée. Tu dessines header / contenu / aside.

```
.galerie {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(240px, 1fr));
  gap: 1rem;
}
```

Une galerie de cartes blog ou boutique : Grid. Essayer la meme chose en Flex + largeurs + wrap fonctionne, mais tu te bats plus pour des colonnes regulieres. Lea a arrete de "forcer des colonnes" en Flex des qu'elle a goute a `auto-fit`.

Les deux ensemble

C'est normal et recommande.

```
<div class="layout">
  <header class="entete">...</header>
  <main>
    <div class="grille-cartes">...</div>
  </main>
</div>
```

`.layout` en Grid (areas). `.entete` en Flex (logo / nav). `.grille-cartes` en Grid (produits). Chaque conteneur choisit son outil. Les enfants ne "melangent" pas les modes du parent : c'est le parent qui decide. Un mode par conteneur. Toujours. Au-dessus de chaque conteneur, un commentaire `/ Grid:` `pLan /` ou `/ Flex: menu /` t'aidera dans six mois.

♦ ASTUCE

Avant de coder, ecris sur papier "Flex" ou "Grid" a cote de trois conteneurs de ta page. Le CSS sort plus court quand le choix est clair.

Cas limites utiles

Une seule colonne de sections qui s'empilent ? Tu n'as pas toujours besoin de Grid. Le flux normal HTML + des marges peut suffire. N'active pas un moteur de layout pour rien.

Un formulaire de deux champs cote a cote sur desktop ? Flex ou Grid, les deux collent. Grid avec `1fr 1fr` est tres clair. Flex avec `flex: 1` aussi. Choisis celui que tu lis le mieux dans six mois.

Un tableau de donnees ? Le HTML `<table>` reste souvent le bon sens semantique. Grid peut mimer un tableau pour des cards, mais pour des vraies donnees tabulaires, garde le tableau.

Signes que tu as choisi le mauvais outil

Tu ajoutes des largeurs en % partout sur des enfants Flex pour "faire des colonnes" et tu pries pour le wrap : regarde Grid. Tu utilises Grid puis tu te bats avec `align-items` pour un simple menu de liens : reviens a Flex. Tu mets `position: absolute` partout pour sauver un layout : souvent un signe que Grid/Flex n'ont pas ete poses. (L'absolu a sa place pour un badge sur image, pas pour toute la page.)

Mini decision rapide

Rangee de controles, menu, barre d'actions → Flex. Page ou galerie en damier → Grid. Les deux niveaux (page + barre) → Grid dehors, Flex dedans (ou l'inverse selon le cas, mais reste coherent).

Ce que ce n'est pas

Ce n'est pas une guerre de camps. Flex n'est pas obsolete. Grid n'est pas "obligatoire partout parce que moderne". Et ce n'est surtout pas déclarer `display: flex` et `display: grid` sur le même élément en espérant un hybride. La dernière déclaration gagne.

⚠ ATTENTION

Un seul mode par conteneur. Si tu empiles `flex` puis `grid` sur le même bloc, tu n'as pas un superpouvoir : tu as une dernière ligne qui écrase l'autre.

Petite histoire

Lea a chronométré une refonte landing : elle a d'abord étiqueté chaque conteneur Flex ou Grid sur papier. Le CSS est sorti plus court, plus clair. Max a vu son neveu se battre avec Flex pour une galerie de photos : passage à Grid, silence soulage. Sam a cassé volontairement une page en mettant les deux `display` sur le header : les élèves ont vu lequel gagnait dans l'inspecteur.

Erreur classique

Déclarer flex et grid sur le même élément. Tout convertir en Grid "parce que c'est moderne". Forcer des colonnes en Flex avec des % partout. Sauver le layout à coups d' `absolute` .

En vrai

Reprenons une page landing. Identifie trois conteneurs. Pour chacun, écris sur un papier "Flex" ou "Grid" et pourquoi. Puis code. Si un choix gêne, switch et note la différence de lisibilité du CSS. L'outil qui produit le CSS le plus court et clair gagne souvent.

Sur une carte produit : essaye le corps de la carte en colonne Flex (`flex-direction: column`) et le footer de carte (prix + bouton) en Flex rangée. La page parente reste en Grid.

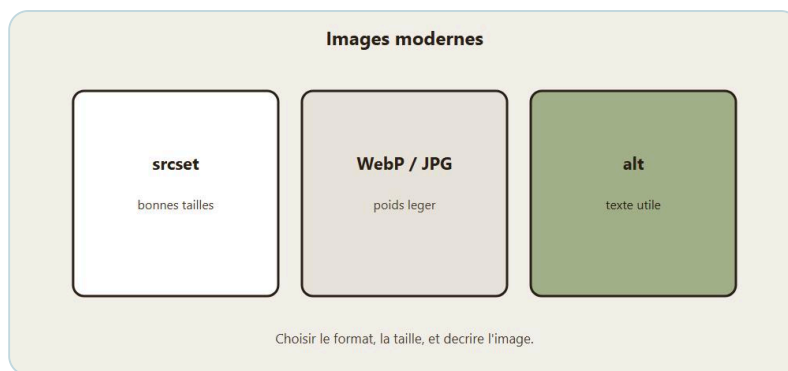
A toi

Construis une mini page "Boutique" : layout page en Grid (entête / grille / pied), entête en Flex, grille de 3 produits en Grid `auto-fit` . Aucun `absolute` pour le plan. Une phrase en commentaire CSS au-dessus de chaque conteneur : `/ Grid: plan /` ou `/ Flex: menu /` .

★ A RETENIR

Flex = une dimension. Grid = lignes + colonnes. Souvent Grid dehors, Flex dedans. Un mode par conteneur.

Chapitre 8 - Images modernes : taille, cadrage, srcset



srcset, format léger, alt utile.

Une belle image mal gérée casse une page. Elle déborde. Elle écrase le texte. Elle pèse 4 Mo et fait attendre le téléphone en 4G. Les bases, tu les as (`img` , `alt`). Ici, on range le comportement CSS et on touche à l'idée de `srcset` .

Chez DanielCraft, une image de produit nette et légère, c'est du respect pour le visiteur. Pas besoin d'être ingénieur perf : quelques réflexes suffisent. Lea vérifie le poids avant chaque livraison. Max a appris après qu'un client a attendu sur 4G. Sam projette l'onglet Réseau : silence, puis "ah".

Ne jamais déborder : max-width

```
img {
  max-width: 100%;
  height: auto;
}
```

Règle d'or. L'image peut retrecir dans son conteneur. Elle garde ses proportions grâce à `height: auto` . Sans ça, une grande photo dans une colonne étroite fait scroller horizontalement. Moche. Sur une carte blog, le conteneur a une largeur ; l'image suit.

Object-fit : cadrer sans déformer

Parfois tu fixes une hauteur de cadre (hero, vignette carrée). L'image source n'a pas le même ratio. `object-fit` décide comment remplir.

```
.carte img {
  width: 100%;
  height: 180px;
  object-fit: cover;
  display: block;
}
```

`cover` : remplit le cadre, recadre ce qui dépasse. Idéal pour des vignettes produit uniformes. `contain` : image entière visible, éventuelles bandes. Utile pour un logo ou un schéma qu'on ne doit pas couper. `fill` : étire (souvent moche). Évite sauf cas spécial.

```
.hero-media img {
  width: 100%;
  height: 320px;
  object-fit: cover;
  object-position: center top;
}
```

`object-position` dit ou ancrer le cadrage (visage en haut, produit centre...).

★ A RETENIR

`max-width: 100% + height: auto` partout. Cadre fixe + `object-fit: cover` pour des vignettes alignées. Le poids, c'est le fichier, pas le CSS.

Le HTML reste responsable du fond

```

```

`width` et `height` (attributs) aident le navigateur a reserver l'espace et limitent le saut de layout pendant le chargement. Le CSS peut ensuite redimensionner. `alt` vide (`alt=""`) seulement si l'image est purement decorative. Sinon, decris ce que l'image apporte, pas "image1".

Idee srcset, en simple

Souvent tu as une grande photo. Sur telephone, telecharger le monstre 2000px est du gaspillage. `srcset` propose plusieurs fichiers ; le navigateur choisit.

```

```

`srcset` liste les fichiers avec leur largeur reelle (`400w` = largeur 400px du fichier). `sizes` dit approximativement quelle largeur l'image occupera a l'ecran. Ici : plein ecran sous 600px, sinon environ 400px (carte dans une grille). `src` reste un fallback.

Tu n'as pas besoin de maitriser tous les cas. Comprends l'idee : plusieurs poids, le navigateur choisit. Pour un projet perso, commencer par compresser une seule image correctement reste deja un gros gain.

`srcset` vient apres.

Formats et poids

JPEG/WebP pour photos. PNG pour schemas a aplats / transparence. Evite d'envoyer un PNG enorme pour une photo. Compresse avant upload (outils en ligne, Squoosh...). Une carte produit a 200 Ko au lieu de 3 Mo, ca se sent. Le CSS ne compresse pas le fichier. Il controle l'affichage. Le poids, c'est le fichier source.

Image dans une grille

```
.grille {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
  gap: 1rem;
}

.carte img {
  width: 100%;
  aspect-ratio: 4 / 3;
  object-fit: cover;
}
```

`aspect-ratio` aide a garder des cartes alignees meme avant chargement complet. Combo sympathique avec `object-fit: cover`.

Ce que ce n'est pas

Ce n'est pas un carrousel de cinq images hero des le premier projet. Une bonne image bien cadre et legere bat un slider bancal. Ce n'est pas non plus `background-image` pour une image informative : moins accessible, plus penible a gerer pour le contenu.

⚠ ATTENTION

Une image de 4 Mo sur une vignette, c'est du mepris pour la 4G. Pese avant de publier. Et n'oublie jamais un `alt` utile sur une image qui informe.

Petite histoire

Lea a livre une galerie avec cinq photos Instagram a pleine resolution. La page a rame sur telephone. Elle a recomprime, unifie les cadres avec `object-fit: cover`, et garde une seule image hero nette. Max a vu la difference sur sa page artisan. Sam projette le panneau Reseau : les eleves comprennent sans discours.

Erreur classique

Mettre une largeur fixe en pixels enorme sur mobile. Oublier `alt`. Forcer `height` sans `object-fit` et ecraser le sujet. Cinq images hero en slider des le premier projet.

En vrai

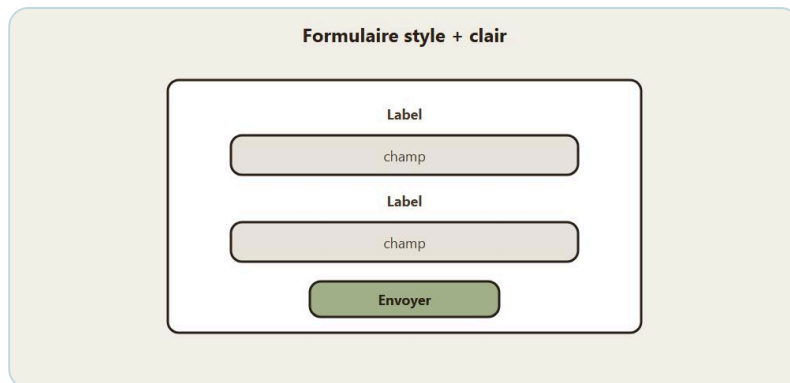
Prends une page produit. Applique `max-width: 100%` globalement sur `img`. Uniformise les vignettes avec hauteur fixe + `object-fit: cover`. Pese ton fichier image. S'il depasse 500 Ko pour une vignette, recomprime.

Si tu as deux tailles de fichier, tente un petit `srcset` sur une seule image et regarde dans l'onglet Réseau quelle variante part selon la largeur de fenêtre.

A toi

Page "Galerie atelier" : grille de quatre images (même fausses URLs ou placeholders). Toutes en `object-fit: cover` dans un cadre `aspect-ratio: 1 / 1`. Un `alt` utile chacune. Bonus : une des images en `srcset` simple a deux largeurs.

Chapitre 9 - Styler un formulaire proprement



Labels clairs, champs lisibles, bouton net.

Tu sais faire un formulaire HTML (champs, labels, bouton submit). La, on le rend agreable et clair. Pas un theme de science-fiction. Un formulaire ou on a envie de taper son email.

Sur une landing, le formulaire d'inscription est souvent le moment de verite. Sur un blog, le contact aussi. Chez DanielCraft, un champ lisible + un bouton evident, ca convertit mieux qu'une animation tape-a-l'oeil. Lea refuse `outline: none` sans remplacement. Max a appris apres qu'un client ne pouvait pas tabber. Sam met un post-it "Tab d'abord" sur les ateliers.

HTML d'abord, toujours

```
<form class="form" action="#" method="post">
  <p>
    <label for="nom">Nom</label>
    <input id="nom" name="nom" type="text" autocomplete="name" required>
  </p>
  <p>
    <label for="email">Email</label>
    <input id="email" name="email" type="email" autocomplete="email" required>
  </p>
  <p>
    <label for="message">Message</label>
    <textarea id="message" name="message" rows="4"></textarea>
  </p>
  <p>
    <button type="submit">Envoyer</button>
  </p>
</form>
```

Label visible, relie avec `for` / `id`. Pas seulement un placeholder. Le placeholder disparaît à la saisie ; le label reste. `autocomplete` aide les navigateurs et les gestionnaires de mots de passe.

Une colonne claire

```
.form {
  display: grid;
  gap: 1rem;
  max-width: 28rem;
}

.form label {
```

```

display: block;
margin-bottom: 0.35rem;
font-weight: 600;
}

.form input,
.form textarea {
width: 100%;
padding: 0.65rem 0.75rem;
border: 1px solid var(--champ-bordure, #ccc);
border-radius: var(--rayon, 8px);
font: inherit;
background: #fff;
color: inherit;
}

```

`font: inherit` évite les champs avec une police système bizarre différente du reste. `width: 100%` dans un conteneur borne, c'est confortable sur mobile.

Focus visible

Quand on tabule jusqu'au champ, il faut voir où on est.

```

.form input:focus,
.form textarea:focus {
border-color: var(--couleur-principale, #1a5f4a);
outline: 2px solid var(--couleur-principale, #1a5f4a);
outline-offset: 2px;
}

```

Ne supprime pas `outline` sans remplacement. C'est un classique d'accessibilité (on y revient au chapitre 11). Chez DanielCraft, le focus visible fait partie du style, pas d'un "plus tard a11y".

⚠ ATTENTION

Un formulaire joli qui perd le focus clavier, c'est un formulaire rate. Placeholder seul n'est pas un label.
`outline: none` global sur tous les inputs non plus.

Bouton coherent

```
.form button {
  border: 0;
  border-radius: var(--rayon, 8px);
  padding: 0.75rem 1.25rem;
  background: var(--couleur-principale, #1a5f4a);
  color: #fff;
  font: inherit;
  cursor: pointer;
}

.form button:hover {
  filter: brightness(1.05);
}

.form button:focus-visible {
  outline: 2px solid var(--couleur-principale, #1a5f4a);
  outline-offset: 3px;
}
```

Le bouton ressemble au reste du site grace aux variables. Pas un gris systeme perdu dans le coin.

Deux champs cote a cote (desktop)

```
.form-rangee {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 1rem;
}

@media (max-width: 600px) {
  .form-rangee {
    grid-template-columns: 1fr;
  }
}
```

Prenom / nom sur desktop, empiles sur mobile. Grid rend ca limpide.

Etats d'erreur (leger)

Sans JavaScript pousse, tu peux deja styler `:invalid` avec prudence (certains navigateurs le montrent tot) :

```
.form input:user-invalid {
  border-color: #a12;
}
```

Si `:user-invalid` n'est pas dispo partout chez toi, garde un style d'erreur via une classe `.champ-erreur` ajoutee plus tard. L'important : le message d'erreur en texte, pas seulement la couleur.

Case a cocher et radio

Laisse assez d'espace cliquable. Associe toujours label et input. Evite de reduire la case a 8px invisibles.


```
.form-check {  
  display: flex;  
  align-items: flex-start;  
  gap: 0.5rem;  
}
```

Flex pour aligner case + texte. Simple.

Ce que ce n'est pas

Ce n'est pas styler uniquement l'état "beau au repos" et oublier hover / focus / disabled. Ce n'est pas non plus un champ mystère design qui ne ressemble plus à un champ. Le tien doit rester évidemment un input.

★ A RETENIR

Labels visibles reliés, champs en pleine largeur dans un conteneur borne, focus clair, bouton de marque. Style = marque + clarté + clavier.

Petite histoire

Lea a livré un formulaire "flottant" sans labels : le client a raté deux champs. Elle a remis les labels au-dessus, le focus visible, et le parcours Tab. Max a fait le même test avec sa page devis. Sam chronomètre le Tab en classe : si un élève hésite, le label n'est pas assez clair.

Erreur classique

Placeholder comme seul label. Champs trop étroits au centre avec du blanc partout sur mobile. Bouton submit sans type clair. Couleur de bordure trop pâle sur fond pâle (contraste).

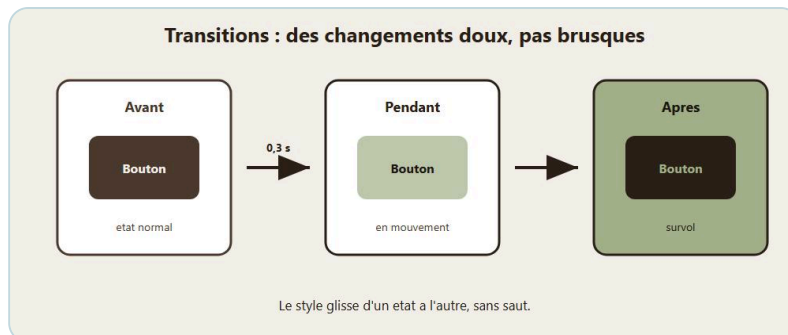
En vrai

Reprends le formulaire contact d'une landing. Passe-le en grille verticale, labels au-dessus, variables de marque, focus visible. Navigue au clavier uniquement. Demande à quelqu'un de le remplir sans explication : s'il hésite, clarifie les labels.

A toi

Formulaire "Inscription atelier" : nom, email, choix de créneau (`select`), message, bouton. Style cohérent avec un `:root` de variables. Rangée prénom/nom en deux colonnes desktop. Test Tab complet. Aucun `outline: none` sans alternative.

Chapitre 10 - Transitions legeres : du mouvement utile



Transitions : un changement doux, pas un spectacle.

Un bouton qui change de couleur d'un coup, c'est correct. Un bouton qui glisse doucement vers sa nouvelle couleur, c'est plus agreable. Les transitions CSS font ca : interpoler entre deux etats quand une propriete change. Tu survoles, le navigateur anime. Tu quittes, il revient. Rien de magique. Juste une consigne de duree et de courbe.

Chez DanielCraft, le mouvement sert la clarte. "Tu survoles, ca reagit." Pas "la page rebondit partout". Lea, freelance web, ajoute 200ms sur ses CTA et ses cartes produit : le client sent que le site est soigne. Max, artisan, veut juste que son bouton devis ne sursaute pas. Sam, enseignant, montre a ses eleves qu'un hover calme vaut mieux qu'un carousel agressif. Trois usages, meme regle : leger, utile, respectueux.

Ce que ce n'est pas

Ce n'est pas une animation permanente. Ce n'est pas un titre qui danse en boucle. Ce n'est pas un pretexte pour `transition: all 2s` sur tout le document. Et ce n'est surtout pas une excuse pour cacher un focus clavier derriere un fondu invisible. La transition embellit un etat clair. Elle ne remplace pas un etat clair.

Etat de base, puis hover

Tu as un etat normal et un etat `:hover` (ou `:focus-visible`). Tu dis au navigateur : "quand ca change, prends 200ms". La propriete `transition` se place en general sur l'etat de base, pas seulement dans `:hover`. Ainsi l'aller et le retour sont doux. Sans ca, l'aller est fluide et le retour est sec - comme une porte qui s'ouvre lentement et claque au retour.

```
.bouton {
  background: var(--couleur-principale);
  color: #fff;
  transition: background-color 200ms ease, transform 200ms ease;
}

.bouton:hover {
  background: #144a3a;
  transform: translateY(-1px);
}
```

♦ ASTUCE

Place `transition` sur `.bouton`, pas seulement dans `:hover`. Tu evites le retour sec et tu gardes un aller-retour coherent.

Que transitionner ?

De bons candidats : `color`, `background-color`, `opacity`, `transform`, `box-shadow` léger, `border-color`.
Des candidats plus risqués : `width`, `height`, `top`, `left` - souvent moins fluides, plus coûteux pour le navigateur. Préférer `transform` et `opacity` quand tu peux.

```
.carte {
  transition: box-shadow 200ms ease, transform 200ms ease;
}

.carte:hover {
  transform: translateY(-2px);
  box-shadow: var(--ombre-hover, 0 8px 20px rgba(0, 0, 0, 0.12));
}
```

Sur une grille de produits, un léger lift au survol guide l'œil sans hurler. Lea l'utilise sur ses cartes boutique. Max s'en passe parfois : un site artisan peut rester très calme et quand même clair.

Durées et courbes

150ms à 300ms pour de l'UI. Au-delà, ça trainasse. En dessous de 100ms, on ne sent presque rien. `ease` et `ease-out` sont naturels pour des hovers. Évite `linear` sur un bouton : ça sonne mécanique. Tu peux lister plusieurs propriétés, ou utiliser `transition: all 200ms ease` avec prudence en proto. En peaufinage, préfère la liste explicite : tu évites de transitionner des choses non voulues.

Liens, focus, et accessibilité du mouvement

```
a {
  color: var(--couleur-principale);
  transition: color 150ms ease;
}

a:hover {
  color: #0f3d30;
}

a:focus-visible {
  outline: 2px solid var(--couleur-principale);
  outline-offset: 2px;
}
```

La transition sur la couleur, oui. Le focus doit rester net : un outline clair, pas uniquement un fondu. Certains reglent leur système pour réduire les animations. Respecte-les.

```
@media (prefers-reduced-motion: reduce) {
  *,
  *::before,
  *::after {
    transition-duration: 0.01ms !important;
    animation-duration: 0.01ms !important;
  }
}
```

Version simple et efficace pour un petit site. Sam l'ajoute systematiquement dans ses demos : le site reste utilisable, sans imposer de mouvement.

★ A RETENIR

Transitions courtes (150-300ms) sur l'etat de base. Hover doux. Focus net. `prefers-reduced-motion` respecte. Calme et net gagne.

Petite histoire

Lea avait mis des transitions de 800ms partout "pour faire pro". Les clients cliquaient avant la fin du fondu. Elle est redescendue a 200ms, a limite aux boutons et cartes, et a ajoute `prefers-reduced-motion`. Le site a soudain paru plus serieux, pas moins. Max, lui, n'avait aucune transition. Son neveu a ajoute un hover sur le bouton devis. Max a garde ca - et rien d'autre. Suffisant.

Erreur classique

Mettre `transition` seulement dans `:hover` : l'aller est doux, le retour est sec. Ou transitionner pendant 2 secondes. Ou animer `width / height` alors qu'un `transform` suffisait. Ou croire que "plus ca bouge, plus c'est moderne". Non.

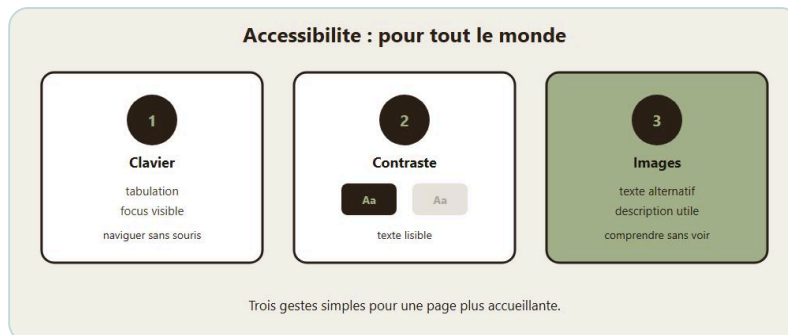
En vrai

Sur une landing, ajoute une transition a : le bouton principal, les cartes, les liens du menu (couleur). Rien d'autre. Navigue a la souris, puis au clavier. Enleve ensuite toutes les transitions et remets-les une par une. Tu sentiras celles qui apportent vraiment. Si ca semble calme et net, c'est gagne.

A toi

Page produit avec une carte et un bouton. Transitions 200ms sur carte (ombre + léger translate) et bouton (fond). Ajoute le media `prefers-reduced-motion`. Verifie hover et focus-visible. Note en une phrase ce que tu as refuse d'animer - ce refus compte autant que ce que tu as anime.

Chapitre 11 - Accessibilite suite : focus, contrastes, labels



Clavier, contraste, texte alternatif.

Le premier livre a planté le décor : textes lisibles, alt, structure. Ici, on solidifie trois réflexes qui changent vraiment la vie des gens : focus visible, contrastes, labels (et un peu d'ordre au clavier).

Accessibilité, ce n'est pas un badge. C'est "est-ce que quelqu'un d'autre que moi peut utiliser cette page sans souffrir ?". Chez DanielCraft, on le traite comme de la qualité produit, pas comme un bonus cosmétique. Lea le fait avant chaque livraison. Max aussi, depuis qu'un client lui a dit "je n'arrive pas à tabber". Sam chronomètre le parcours Tab en classe.

Focus visible : savoir ou on est

Beaucoup de gens naviguent au clavier (Tab, Shift+Tab, Entrée). Si tu enlèves les contours de focus parce que "c'est moche", tu les perds dans la page.

```
:focus-visible {
  outline: 2px solid var(--couleur-principale, #1a5f4a);
  outline-offset: 2px;
}
```

`:focus-visible` cible surtout le focus clavier (selon navigateurs / contexte), ce qui évite parfois un anneau au simple clic souris. Tu peux affiner par composant (boutons, liens, champs).

```
.bouton:focus-visible {
  outline: 3px solid var(--couleur-principale);
  outline-offset: 3px;
}
```

Test minimal : charge ta page, cache la souris, Tab jusqu'au bout. Tu dois toujours voir clairement l'élément actif.

★ A RETENIR

Focus visible, contrastes solides, labels reliés. Tester au clavier. Ce n'est pas un chapitre "en plus" : c'est la qualité de la page pour de vrai monde.

Contrastes : lisible, pas juste joli

Texte gris pale sur fond beige, ca fait "design soft"... et illegible au soleil ou avec une vue fatiguee. Vise un contraste solide entre texte et fond. Texte courant sombre sur fond clair, ou texte clair sur fond sombre. Les liens ne se distinguent pas seulement par la couleur (souligne au hover/focus, ou poids). Les placeholders tres clairs ne remplacent pas un label.

Tu peux verifier avec un outil de contraste (extensions, sites de check). Si tu hesites entre deux beiges, choisis le plus lisible. La marque survit a un contraste correct.

```
:root {  
  --couleur-texte: #1b1b1b;  
  --couleur-fond: #f7f5f0;  
  --couleur-muette: #444;  
}
```

`--couleur-muette` trop claire (#aaa sur blanc) devient un piege pour les mentions legales.

Labels : dire le nom des champs

On l'a vu au chapitre formulaires. On insiste parce que c'est le bug n°1 des "jolis" formulaires.

```
<label for="email">Email</label>  
<input id="email" type="email" name="email">
```

Pas ca :

```
<input type="email" placeholder="Email">
```

Le placeholder peut aider en exemple (`nom@domaine.fr`), pas en remplacement du label.

Pour une case a cocher :

```
<label>  
  <input type="checkbox" name="newsletter">  
  J'accepte de recevoir la newsletter  
</label>
```

Ou `for` / `id` separees. L'important : le nom est annonce clairement.

Ordre de tabulation naturel

L'ordre du HTML guide le Tab. Si tu deplaces visuellement des blocs avec Grid/Flex, l'ordre visuel peut diverger de l'ordre clavier. En cas de doute, rearrange le HTML pour coller a l'ordre de lecture logique (entete, contenu, aside...).

Evite `tabindex` positifs inventes (1, 2, 3...) : ca devient un labyrinthe. `tabindex="0"` parfois pour rendre un element focusable ; `tabindex="-1"` pour du focus programme. Au debut, le HTML bien ordonne suffit.

Images et boutons

Image informative → `alt` utile. Bouton icône seul → nom accessible (texte visible, ou `aria-label` si vraiment icône seule).

```
<button type="button" aria-label="Fermer">X</button>
```

Mieux encore : un texte "Fermer" visible si tu as la place.

Landing : checklist express

Un seul `h1`. Menu atteignable au clavier. CTA avec focus visible. Contraste du texte hero (attention texte blanc sur photo claire : ajoute un voile sombre). Formulaire avec labels.

```
.hero {  
  background: linear-gradient(rgba(0, 0, 0, 0.45), rgba(0, 0, 0, 0.45)),  
    url("hero.jpg") center/cover;  
  color: #fff;  
}
```

Le dégradé aide le contraste du texte sur image.

Ce que ce n'est pas

Ce n'est pas croire que "personne ne navigue au clavier". Faux. Et même parmi les souris, certains ont besoin de focus clair. Ce n'est pas non plus un badge "accessible" sans test.

⚠ ATTENTION

`outline: none` global. Liens oranges sur fond orange. Formulaire "flottant" sans labels. Ces trois pièges cassent une page "jolie" en page inutilisable.



Exemple : naviguer au clavier avec un focus visible.

Petite histoire

Lea a Tabbe une landing client juste avant envoi : le CTA disparaissait au focus. Elle a remis un outline net et un voile sur le hero. Max a fait le même geste sur sa page devis après le retour d'un client. Sam active Narrateur cinq minutes en classe : les élèves entendent les trous.

Erreur classique

`outline: none` global. Liens trop pales. Formulaire sans labels. Modale ou menu qui piège le focus (sujet avance : au moins, ne casse pas Tab sur une page simple).

En vrai

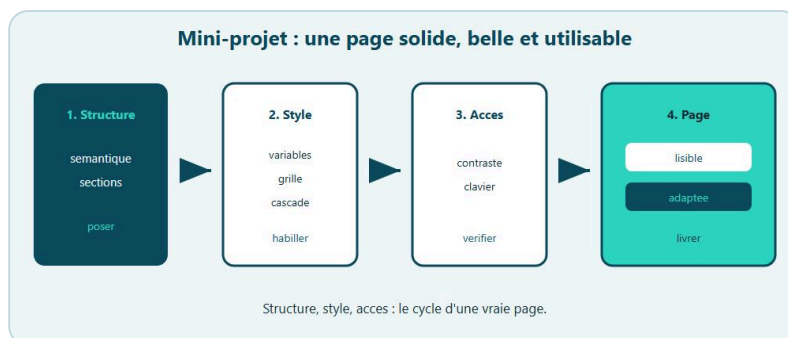
Prends ta page produit ou landing. Fais le parcours Tab complet. Note chaque trou (focus invisible, ordre bizarre). Monte le contraste d'un texte secondaire trop pale. Verifie chaque `input` pour un label.

Si tu peux, active un lecteur d'ecran basique juste cinq minutes (Narrateur sur Windows, VoiceOver sur Mac) et ecoute la page. Tu entendras vite les trous.

A toi

Ameliore une page existante avec trois commits mentaux : (1) `:focus-visible` global propre, (2) correction de deux contrastes faibles, (3) labels complets sur un formulaire. Ecris en trois lignes ce que tu as change. C'est ton "rapport a11y" personnel. Chez DanielCraft, ce petit rapport vaut plus qu'un badge sans test.

Chapitre 12 - Mini-projet : page d'accueil responsive (Grid + variables)



Mini-projet : page d'accueil responsive.

On assemble. Tu vas construire une petite page d'accueil : marque, menu, hero, grille de cartes, bandeau d'appel, formulaire court, pied. Responsive. Theme via variables. Layout via Grid. Pas de framework.

L'objectif n'est pas la perfection pixel. C'est une page coherente, lisible, utilisable au clavier, qui tient sur telephone et desktop. Le genre de base qu'on aimerait livrer pour un atelier, une boutique naive, ou une vitrine DanielCraft miniature. Lea pose toujours la question "c'est pour quoi, en cinq secondes ?". Max aussi. Sam chronometre la reponse en classe.

Brief

Sujet libre, mais concret. Exemples : "Atelier ceramique", "Cafe de quartier", "Coach sportif", "Librairie".

La page contient un header avec nom + navigation (2-4 liens), un hero avec `h1`, une phrase, un bouton, une section "Nos pepites" (ou equivalent) : grille de 3 ou 4 cartes (titre, texte, lien), une section contact : petit formulaire (nom, email, message), un footer simple.

Sur grand ecran : hero confortable, grille multi-colonnes. Sur petit ecran : tout s'empile, boutons aises a taper.

Etape 1 - HTML semantique

Pose le squelette sans te battre avec le style.

```

<body>
  <header class="site-header">...</header>
  <main>
    <section class="hero">...</section>
    <section class="section" id="cartes">...</section>
    <section class="section" id="contact">...</section>
  </main>
  <footer class="site-footer">...</footer>
</body>
  
```

Un `h1` dans le hero. Des `h2` de section. Cartes en `article`. Formulaire avec labels. Liens du menu qui pointent vers `#cartes` et `#contact` si tu veux du one-page.

Etape 2 - Variables

```
:root {
  --couleur-principale: #1a5f4a;
  --couleur-fond: #f7f5f0;
  --couleur-texte: #1b1b1b;
  --couleur-carte: #fff;
  --rayon: 10px;
  --espace: 1rem;
  --gap: 1.25rem;
  --largeur-max: 1100px;
  --ombre: 0 4px 14px rgba(0, 0, 0, 0.08);
}
```

Branche `body`, liens, boutons, cartes sur ces variables. Aucune couleur magique éparpillée si tu peux l'éviter.

★ A RETENIR

Theme 100 % variables, grille en auto-fit, HTML sémantique, formulaire labels, Tab impeccable.
Livrable imparfait > concept parfait non code.

Etape 3 - Coquille et header

```
body {
  margin: 0;
  font-family: Georgia, "Times New Roman", serif;
  background: var(--couleur-fond);
  color: var(--couleur-texte);
  line-height: 1.5;
}

.wrap {
  max-width: var(--largeur-max);
  margin-inline: auto;
  padding: var(--espace);
}

.site-header {
  display: flex;
  justify-content: space-between;
  align-items: center;
  gap: var(--espace);
  flex-wrap: wrap;
}
```

Flex pour le header. Serif volontaire pour sortir des stacks par défaut, tout en restant simple. Tu peux choisir une autre police web si tu veux, tant que c'est lisible.

Etape 4 - Hero

```
.hero {
  display: grid;
  gap: var(--espace);
  padding: calc(var(--espace) * 2) 0;
}

.hero h1 {
  font-size: clamp(1.8rem, 4vw, 3rem);
  line-height: 1.15;
  margin: 0;
}
```

`clamp` donne une taille fluide entre min et max. Pas obligatoire, mais agreable. Un seul CTA principal dans le hero.

Etape 5 - Grille de cartes

```
.grille {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
  gap: var(--gap);
}

.carte {
  background: var(--couleur-carte);
  border-radius: var(--rayon);
  padding: var(--espace);
  box-shadow: var(--ombre);
}
```

Images optionnelles : si tu en mets, `max-width: 100%` + `object-fit: cover` + `aspect-ratio`.

Etape 6 a 8 - Formulaire, transitions, a11y

Reprend les patterns du chapitre 9. Formulaire en grid verticale, max-width raisonnable. Footer discret avec contraste correct. Boutons et cartes : transitions 200ms. `prefers-reduced-motion` en bas de fichier. Tab complet. Focus visible. Labels. Contraste hero. `alt` si images.

Criteres de reussite

La page se lit sans CSS (ordre logique). Changer 2 variables change le theme de facon visible. La grille passe d'une a plusieurs colonnes sans casser. Pas de scroll horizontal sur mobile. Focus visible sur liens, boutons, champs.

Ce que ce n'est pas

Ce n'est pas tout faire en `position: absolute`. Ce n'est pas copier un theme enormement complexe. Ce n'est pas cinq CTA aussi forts les uns que les autres dans le hero.

⚠ ATTENTION

Grille fixe `1fr 1fr 1fr` sans media query écrase le téléphone. Préfère `auto-fit` + `minmax`, ou un breakpoint qui repasse en une colonne.

Petite histoire

Lea a chronométré 90 minutes pour une première version "atelier céramique". Livrable imparfait, mais cohérent. Elle a montré la page à un ami : "c'est clair, j'irais". Max a regardé la même page et a dit "je trouve le prix, je trouve le bouton". Sam valide dès que le Tab passe - même si une carte est encore un peu pâle.

Erreur classique

Oublier le formulaire labels. Mettre cinq CTA dans le hero. Grille fixe sans media query. Tout peaufiner jusqu'à minuit sans jamais montrer.

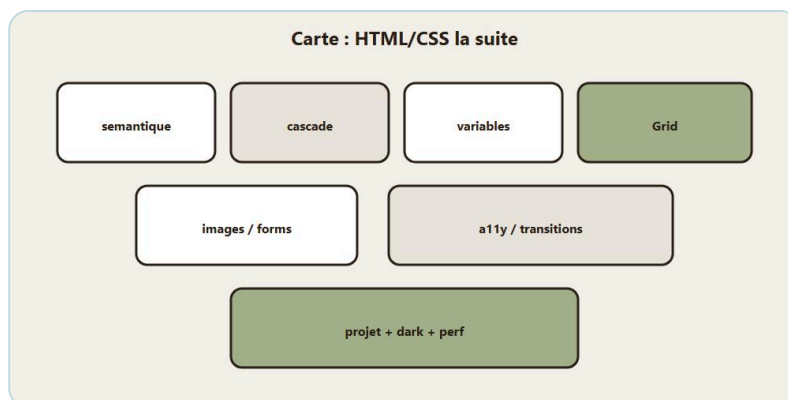
En vrai

Chronomètre-toi 90 minutes. Livrable imparfait > concept parfait non code. Ensuite tu peaufines 30 minutes : contrastes, gaps, textes. Montre la page à quelqu'un. Demande : "C'est pour quoi, en cinq secondes ?" Si la réponse matche ton intention, le hero fait son job.

A toi

Livre `index.html` + `styles.css`. Nom du projet dans le header. Trois cartes minimum. Un formulaire. Un thème 100 % variables. Bonus : une variante en commentant une autre palette dans `:root` (tu basculeras vraiment au chapitre dark mode). Chez DanielCraft, oser montrer bat peaufiner seul jusqu'à minuit.

Chapitre 13 - A retenir



Carte de la suite HTML/CSS.

Tu as parcouru pas mal de terrain. Avant les ateliers et les chapitres d'affutage, on pose les pièces sur la table. Pas une encyclopédie. Une checklist mentale en paragraphes, pour que ça tienne quand tu codes sans notes. Chez DanielCraft, on préfère cinq réflexes solides à cinquante astuces oubliées.

Imagine une page produit, un petit blog, une landing. Tu ouvres le fichier. Tu te demandes, dans l'ordre : la structure a-t-elle un sens ? Qui gagne en CSS ? Les couleurs viennent-elles d'un seul endroit ? Le plan tient-il en large et en étroit ? Les images et le formulaire sont-ils accueillants ? Le mouvement est-il calme ? Le clavier passe-t-il ? Si tu réponds oui à ça, tu es déjà au niveau "suite", pas seulement "bases".

Lea résume ça en une phrase pour ses clients : "propre, cohérent, utilisable". Max le vit autrement : "je retrouve mon bouton devis sur téléphone". Sam le dit à ses élèves : "si tu expliques ta page à voix haute avec les balises, tu as gagné".

Structure et cascade

Le HTML sémantique nomme les pièces : `header`, `nav`, `main`, `article`, `section`, `footer`, parfois `aside`. Les `div` restent pour le layout neutre. Un `main`, un `h1` fort. Sans ça, ta page est un tas de boîtes anonymes - le navigateur et les lecteurs d'écran galèrent, et toi aussi dans six mois.

À poids égal, la dernière règle gagne. Les classes pèsent plus que les types. Les ID pèsent encore plus : évite-les pour le style quotidien. **!important** presque jamais. L'inspecteur montre qui gagne. Quand une couleur "ne change pas", ce n'est pas de la magie noire : c'est de la spécificité. Relis le chapitre 3 si besoin, mais garde le réflexe inspecteur.

Variables, Grid, Flex

`:root` pour la palette et les rythmes (`--couleur-principale`, `--espace`, `--rayon`...). `var(--nom)` partout. Une variable peut se redéfinir localement dans un bloc. Changer le thème, c'est d'abord changer le `:root`. Lea ne chasse plus les hex dans cinquante classes : elle touche trois variables et la marque bascule.

Grid : `display: grid`, `grid-template-columns`, `gap`, `repeat`, `minmax` / `auto-fit` pour des galeries. `grid-template-areas` pour une page header / contenu / aside / pied. Placement simple avec `grid-column` si besoin. Flex : une dimension - menus, barres, alignements. Grid : deux dimensions - plan de page, damier. Souvent Grid dehors, Flex dedans. Un mode par conteneur. Ce n'est pas une guerre de camps : c'est une boîte à outils.

★ A RETENIR

Sens (HTML), marque (:root), plan (Grid/Flex), detail utile, puis verification (a11y, Tab). Les mots aident. Les pages livrees prouvent.

Images, formulaires, mouvement, accessibilite

Images : `max-width: 100%` + `height: auto` . `object-fit: cover` pour des cadres. `alt` utile. Poids du fichier soigne. `srcset` comme idee pour servir la bonne largeur. Formulaires : labels visibles relies, champs en pleine largeur dans un conteneur borne, focus clair, bouton de marque. Deux colonnes en Grid sur desktop si utile, une sur mobile.

Mouvement : transitions courtes sur l'etat de base, hover doux, `prefers-reduced-motion` respecte. Pas de cirque. Accessibilite : focus visible, contrastes, labels, ordre Tab logique. Voile sombre si texte sur photo. Tester au clavier. Ce n'est pas un chapitre "en plus" : c'est la qualite de la page pour de vrai monde.

Ce que ce n'est pas

Ce n'est pas une liste a recracher au quiz sans avoir code. Ce n'est pas non plus "je connais les mots donc je sais faire une page pro". Le mini-projet vise une home responsive qui assemble variables + Grid + semantique + formulaire + transitions legeres. Les ateliers qui suivent font faire. Dark mode, perf et debug solidifient.

⚠ ATTENTION

Sauter directement aux ateliers sans ce recap, puis melanger trois approches de layout "parce que ca marchait sur Stack Overflow", c'est le piege. Les reflexes restent quand le framework change. Ici, HTML et CSS nus.

Petite histoire

Lea a refait une landing client en une soiree en suivant exactement cette checklist. Elle a trouve trois verts differents, un `div` menu, une image a 2 Mo, et zero focus. En deux heures, variables, semantique, Grid, image recompressée, outline de focus. Le client a dit "ca fait plus cher". C'etait juste plus propre. Max a demande a son neveu la meme checklist sur sa page artisan : le devis mobile a cesse de debord. Sam a transforme la checklist en oral de cinq minutes en classe. Ca marche.

Erreur classique

Croire qu'un framework remplacera ces reflexes. Ou sauter le recap et tout melanger ensuite. Ou confondre "je connais les mots" et "je sais livrer".

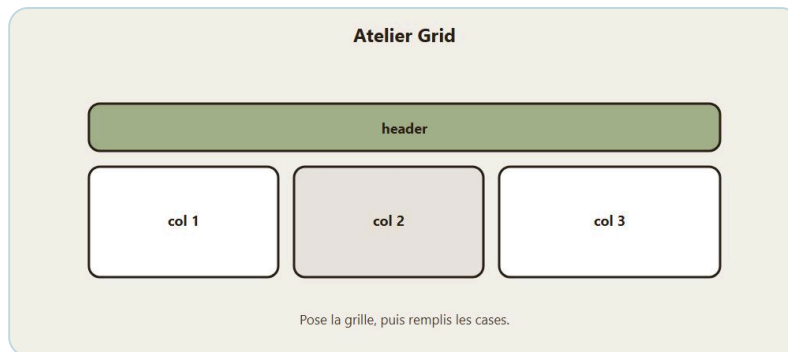
En vrai

Sans regarder tes notes, ouvre une page que tu as codee cette semaine. Coche mentalement : semantique, cascade comprise, variables, Grid ou Flex choisi, images, formulaire, transitions, a11y. Note les trous. Les ateliers sont la pour les boucher. Chronometre cinq minutes. Pas plus. La carte doit tenir sans roman.

A toi

Ecris dix lignes : cinq choses que tu sais faire maintenant, cinq pieges que tu veux eviter. Garde ce papier a cote des ateliers. Chez DanielCraft, ce bout de papier vaut plus qu'un screenshot de quiz reussi. Bonus : montre-le a un ami et demande ce qui manque pour "oser envoyer la page".

Chapitre 14 - Atelier Grid : blog en zones



Atelier : poser la grille, puis remplir.

Atelier pratique. Objectif : construire le squelette d'un petit blog en Grid avec `grid-template-areas`, puis l'adapter en une colonne sur petit écran. Pas de framework. HTML + CSS. Tu peux réutiliser des couleurs via des variables. Chez DanielCraft, on aime les ateliers qui laissent une page sous la main, pas seulement une théorie correctement recopiée.

Lea fait ce genre de squelette avant chaque refonte blog client : zones d'abord, deco ensuite. Max n'a pas de blog, mais il comprend l'idée quand on lui montre "entête / contenu / aside / pied" comme les pièces d'un atelier. Sam donne exactement ce brief à ses élèves : un plan lisible avant le pixel perfect.

Ce que ce n'est pas

Ce n'est pas un concours de CSS créatif. Ce n'est pas non plus "copier un layout GitHub sans comprendre les noms de zones". Et ce n'est pas l'endroit pour `position: absolute` sur le plan général. Si tu triches avec du positionnement absolu pour "faire tenir" le blog, tu rates l'atelier même si ça a l'air joli une minute.

Brief

La page contient un en-tête (titre du blog + menu), un contenu principal avec deux articles, une colonne "À lire aussi" avec trois liens, un pied. Sur grand écran : contenu et aside côte à côte, entête et pied pleine largeur. Sur petit écran : tout s'empile dans un ordre lisible - en-tête, contenu, aside, pied. Thème libre : "Notes de voyage", "Cuisine du dimanche", "Journal d'atelier"...

Tu dessines d'abord le plan sur papier en deux cases (large / étroit). Ensuite seulement tu codes. Le papier évite la panique devant `grid-template-areas`.

Étapes

1. Écris le HTML sémantique : `header`, `nav`, `main` avec deux `article`, `aside`, `footer`.
2. Crée un conteneur `.layout` en `display: grid`.
3. Dessine les areas sur grand écran (entête pleine largeur, contenu + aside, pied pleine largeur).
4. Relie chaque pièce avec `grid-area`.
5. Ajoute `gap` et une largeur max centrée (`max-width` + `margin-inline: auto`).
6. Déclare un `:root` minimal (couleurs, gap).
7. Sous `max-width: 700px` (ou similaire), repasse en une colonne et redéfinit `grid-template-areas` pour empiler.

8. 8. Style léger des articles (fond carte, rayon) sans casser le plan.

Amorce CSS

```
:root {
  --couleur-principale: #1a5f4a;
  --fond: #f7f5f0;
  --carte: #fff;
  --gap: 1.5rem;
}

.layout {
  display: grid;
  grid-template-columns: 2fr 1fr;
  grid-template-areas:
    "entete entete"
    "contenu aside"
    "pied pied";
  gap: var(--gap);
  max-width: 1000px;
  margin-inline: auto;
  padding: 1rem;
}

.entete { grid-area: entete; }
.contenu { grid-area: contenu; }
.aside { grid-area: aside; }
.pied { grid-area: pied; }

@media (max-width: 700px) {
  .layout {
    grid-template-columns: 1fr;
    grid-template-areas:
      "entete"
      "contenu"
      "aside"
      "pied";
  }
}
```

Adapte si tu ajoutes une colonne menu. L'important : le dessin reste coherent (meme nombre de cellules par ligne). Une faute de typo dans un nom de zone et toute la grille "glisse" : l'outline temporaire (chapitre debug) t'aidera.

♦ ASTUCE

Colorie temporairement chaque area (fond different) pour voir qui est ou. Enleve apres. C'est le meilleur projecteur quand tu bloques.

Contenu minimum et criteres

Chaque article : un `h2`, un paragraphe, un lien "Lire la suite". L'aside : un titre et une liste de liens. Pas besoin de vrais articles longs. Un `h1` dans l'entete ou au debut du `main` (un seul).

Criteres de reussite : les zones sont au bon endroit ; le HTML se lit sans CSS dans la tete (ordre logique) ; sur mobile, rien ne debord ; tu n'as pas triche avec des `absolute` pour le plan general ; les variables pilotent au moins fond et gap.

Petite histoire

Sam a vu un eleve mettre l'aside avant le `main` dans le HTML "pour aider le CSS desktop". Sur mobile, au Tab, l'aside arrivait avant les articles. Le correctif : ordre de lecture contenu puis aside, et laisser Grid placer. Lea a eu le meme reflexe sur un projet client. Max, en voyant la demo, a dit : "c'est comme ranger l'atelier pour que le client trouve le devis avant les factures fournisseurs". Exactement.

⚠ ATTENTION

Oublier de redefinir `grid-template-areas` dans le media query te laisse avec deux colonnes ecrasees sur telephone. Chaque breakpoint a son dessin.

Erreur classique

Copier un layout Grid trouve sur le web sans comprendre les noms de zones. Typo dans un nom de zone. Ordre HTML qui place l'aside avant le contenu "pour aider le CSS".

En vrai

Chronometre 90 minutes. Premiere heure : HTML + grille qui tient. Demi-heure : style léger et test mobile. Si au bout d'une heure le plan tient sans deco, tu as deja gagne l'essentiel. La deco vient apres, jamais avant le plan.

Bonus

Une carte "article epingle" avec `grid-column: 1 / -1` au-dessus des deux articles dans le `main` (petite grille interne). Ou un header en Flex (titre / nav). Optionnel. Ne saute pas dessus tant que le squelette n'est pas valide.

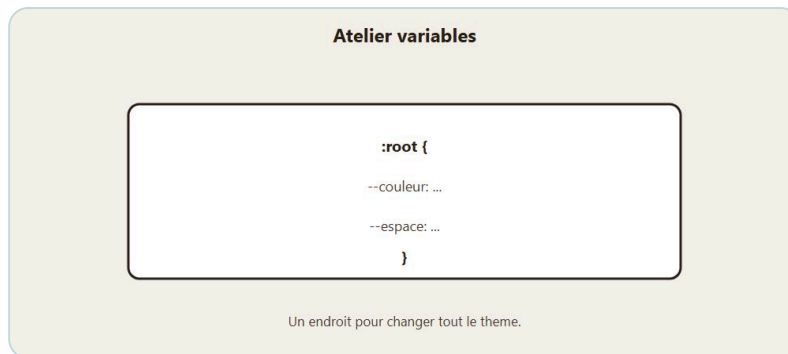
A toi

Livre la page. Change le texte des articles. Si le layout tient en large et en etroit, l'atelier est valide. Montre-la a quelqu'un : demande ou est "le menu" et "l'article" en trois secondes. Note la reponse. Si la personne hesite, ton plan n'est pas encore assez clair - et c'est utile a savoir.

★ A RETENIR

`grid-template-areas` = plan en mots. Meme nombre de cellules par ligne. Contenu avant aside dans le HTML. Une colonne sur mobile.

Chapitre 15 - Atelier variables : une marque en `:root`



Atelier `:root` pour piloter le theme.

Atelier pratique. Objectif : themater une petite page produit uniquement via des variables CSS. Tu dois pouvoir changer toute l'ambiance en touchant `:root`, sans chasser des couleurs en dur dans cinquante classes. Chez DanielCraft, c'est le geste qui separe "page bricolee" et "page de marque".

Lea refuse desormais de livrer une carte produit sans variables : un client qui change d'avis sur le vert ne doit pas couter une soiree de chercher-remplacer. Max comprend l'idee quand on lui dit "une peinture pour tout l'atelier, pas un pot par mur". Sam fait basculer le theme en cours devant la classe : les eleves voient le pouvoir du `:root` en une seconde.

Ce que ce n'est pas

Ce n'est pas encore le dark mode systeme du chapitre 17 (meme si tu t'en approches). Ce n'est pas non plus "mettre deux variables et garder le reste en hex". Et ce n'est pas un concours de gradients. Une palette simple, lisible, coherente. Si tu triches avec un `#1a5f4a` "temporaire" dans `.bouton`, le temporaire reste - et l'atelier rate son but.

Brief

Page "Carte produit" : nom de boutique, une carte (titre produit, court texte, prix, bouton), trois pastilles d'info en dessous (livraison, retour, origine). Tu livres deux palettes : une claire, une "soir" (plus sombre). La bascule peut etre manuelle : tu commentes / decommentes un bloc `:root`, ou - mieux - tu ajoutes une classe sur `body`. Quand tu bascules, tout suit. Si quelque chose reste fige, c'est qu'un hex trainee encore dans une classe.

Etapes

1. Pose le HTML semantique minimal (`header`, `main`, `article`, `footer`).
2. Cree un `:root` avec au moins : `--couleur-principale`, `--fond`, `--texte`, `--carte`, `--rayon`, `--espace`, `--ombre`.
3. Branche body, carte, liens, bouton, pastilles sur `var(...)`.
4. Zero couleur hex "en dur" hors de `:root` (sauf eventuel blanc/noir declares aussi en variables).
5. Cree une seconde palette.
6. Fais basculer via `body.theme-soir` (recommande) ou en echangeant le bloc `:root`.
7. Verifie les contrastes dans les deux themes.
8. Ajoute une transition legere sur le bouton (toujours via variables pour la couleur).

Amorce palette + bascule

```
:root {
  --couleur-principale: #1a5f4a;
  --fond: #f7f5f0;
  --texte: #1b1b1b;
  --carte: #ffffff;
  --rayon: 10px;
  --espace: 1rem;
  --ombre: 0 4px 14px rgba(0, 0, 0, 0.08);
}

body.theme-soir {
  --couleur-principale: #5dcaa5;
  --fond: #14221c;
  --texte: #eef5f1;
  --carte: #1c3229;
  --ombre: 0 4px 14px rgba(0, 0, 0, 0.4);
}

body {
  background: var(--fond);
  color: var(--texte);
}

.carte {
  background: var(--carte);
  border-radius: var(--rayon);
  padding: calc(var(--espace) * 1.25);
  box-shadow: var(--ombre);
}

.bouton {
  background: var(--couleur-principale);
  color: var(--fond);
  border: 0;
  border-radius: var(--rayon);
  padding: 0.7rem 1.1rem;
  transition: filter 200ms ease;
}
```

Pour tester le theme soir : `<body class="theme-soir">` . Au chapitre dark mode, tu brancheras une version plus "systeme". Ici, l'important est la discipline des variables.

★ A RETENIR

Tout le theme passe par des variables. Changer `:root` (ou une classe sur `body`) change fond, texte, carte, bouton. Zero hex fugitif dans les classes.

Pastilles et criteres

Une petite grille `repeat(3, 1fr)` sur desktop, une colonne sur mobile. Chaque pastille utilise `--carte` et une bordure simple en `var(--couleur-principale)` .

```
.infos {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(140px, 1fr));
  gap: var(--espace);
}
```

Criteres de reussite : changer uniquement les variables change fond, texte, carte, bouton ; les deux themes restent lisibles (contraste) ; pas de chasse au hex dans les classes de composants ; le HTML reste simple et semantique.

Petite histoire

Lea a livre une page avec theme clair uniquement. Le client a demande "version nuit pour Instagram Stories". Elle a ajoute `body.theme-soir` en vingt minutes parce que tout etait deja en variables. La fois d'avant, sans variables, ca avait pris trois heures et casse deux contrastes. Max a vu la demo et a demande la meme chose pour sa page devis - "le soir, mes clients lisent sur le canape". Sam a note le cas pour le cours suivant.

⚠ ATTENTION

Theme sombre avec texte `#666` sur fond `#14221c` : illisible. Monte le contraste. Un theme "joli" mais illisible n'est pas un theme : c'est une deco dangereuse. Et le `#1a5f4a` "temporaire" dans `.bouton` reste presque toujours.

Erreur classique

Garder des hex en dur hors de `:root`. Contrastes insuffisants en theme soir. Croire que deux variables suffisent pendant que le reste du CSS reste en dur.

En vrai

Fais la chasse aux hex : cherche `#` dans ton CSS. Tout ce qui n'est pas dans `:root` ou `body.theme-soir` est suspect. Corrige. Bascule dix fois de suite. Si un element clignote encore dans l'ancienne couleur, tu as trouve le fugitif.

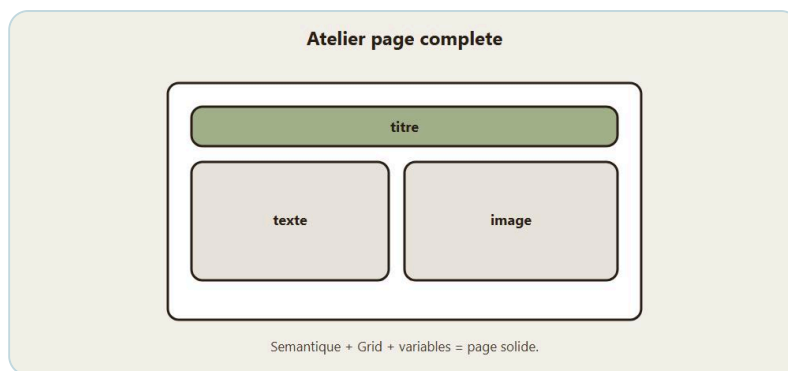
Bonus

Ajoute `--font-titre` et `--font-corps` en variables (noms de familles). Ou un `--accent` pour le prix uniquement. Utile, pas obligatoire.

A toi

Livre la page + les deux themes. Fais une capture mentale (ou reelle) des deux versions. Si tu peux basculer en une classe, tu es pret pour le chapitre dark mode. Ecris en une phrase la couleur que tu as du eclaircir ou foncer pour le contraste - ce geste-la, c'est le metier.

Chapitre 16 - Atelier page : assembler une mini landing



Atelier : assembler une page complete.

Atelier pratique. Objectif : produire une landing one-page complete, propre, responsive, en collant sémantique + variables + Grid + formulaire + transitions légères + focus visible. C'est le cousin guide du mini-projet. Si tu as déjà une home du chapitre 12, tu peux la refondre ici avec un brief plus strict. Sinon, pars de zéro.

Chez DanielCraft, une landing réussie tient en une promesse claire, un parcours simple, et une inscription sans friction. Lea en livre souvent pour des ateliers clients. Max en aurait besoin pour un stage "plomberie du dimanche" s'il se lançait. Sam en fait construire une par binôme en fin de module. Même squelette, trois vies.

Ce que ce n'est pas

Ce n'est pas un site multi-pages. Ce n'est pas non plus le moment d'ajouter témoignages, logos partenaires, FAQ, fil Instagram et chat flottant. Socle d'abord. Autre chose que ce n'est pas : deux boutons dans le hero de même poids visuel. Un CTA principal suffit. Si tout crie, rien ne guide.

Brief obligatoire

Landing pour un atelier d'une journée (sujet libre : photo, pain, couture, code...). Tu arrives, tu comprends l'offre en cinq secondes, tu vois le programme, tu compares deux tarifs, tu t'inscris. Sur téléphone, rien ne casse. Au Tab, tu sens chaque étape. Les couleurs viennent du `:root`. Si tu changes `--couleur-principale`, la marque suit.

Sections : (1) Header (marque + nav ancres), (2) Hero (promesse + CTA), (3) Programme (3 étapes en grille), (4) Tarifs (2 cartes côte à côte sur desktop), (5) Inscription (formulaire), (6) Footer. Contraintes : pas de framework, pas d'`absolute` pour le plan général, thème 100 % variables, Tab impeccable.

Étapes

1. Écris tout le HTML d'un trait, textes provisoires OK.
2. Pose `:root` (palette + espace + rayon + largeur max).
3. Style le header en Flex.
4. Hero en Grid simple (texte ; optionnellement une image à côté sur desktop).
5. Section programme en `auto-fit` / `minmax`.
6. Section tarifs en `1fr 1fr` puis `1fr` sous 600px.

7. 7. Formulaire labels + focus (chapitre 9).
8. 8. Transitions 200ms sur CTA et cartes.
9. 9. Media `prefers-reduced-motion`.
10. 10. Passe le parcours Tab et corrige.
11. 11. Teste a 320px, 768px, 1200px de large (ou redimensionne a la main).

Amorce hero + tarifs

```
.hero {
  display: grid;
  gap: 1.5rem;
  align-items: center;
}

@media (min-width: 800px) {
  .hero {
    grid-template-columns: 1.2fr 1fr;
  }
}

.tarifs {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: var(--gap);
}

@media (max-width: 600px) {
  .tarifs {
    grid-template-columns: 1fr;
  }
}
```

Si tu mets une image hero : `object-fit: cover`, `alt` utile, poids raisonnable. Un voile de contraste si texte sur photo.

★ A RETENIR

Une promesse claire, un parcours simple, une inscription sans friction. Theme 100 % variables. Tab impeccable. Socle avant les "nice to have".

Contenu minimum et criteres

Hero : une phrase nette ("Apprends X en une journee"). Programme : trois titres courts. Tarifs : "Solo" et "Duo" (ou equivalent) avec prix et liste courte. Formulaire : nom, email, choix de date (`select`), bouton.

Criteres : on comprend l'offre en cinq secondes ; la page ne casse pas en etroit ; les ancrs du menu amenant aux sections ; le formulaire est utilisable au clavier ; changer `--couleur-principale` change clairement la marque.

Petite histoire

Lea a chronometre deux heures chrono pour une premiere version "atelier photo argentique". La premiere heure a tout pose. La seconde a corrige contrastes, gaps, textes. Elle a envoye a un ami : "c'est clair, j'irais". Max a regarde la meme page et a dit "je trouve le prix, je trouve le bouton". Sam a valide le binome des que le Tab passait sans accroc - meme si une carte etait encore un peu pale. Les priorites comptent.

⚠ ATTENTION

Trop de sections "nice to have" avant d'avoir le socle. Hero a 3 Mo. Formulaire sans labels "parce que le placeholder suffit". Ancres cassees parce que les `id` ne matchent pas. Verifie chaque lien du menu a la main.

Erreur classique

Deux CTA de meme poids dans le hero. Peaufiner des ombres pendant trois heures sans tester le parcours. Oublier `prefers-reduced-motion`.

En vrai

Coupe le Wi-Fi mental des reseaux sociaux. Met un timer 2 heures. Premiere version uniquement. Ensuite 20 minutes de polish. Si tu depasses trois heures a peaufiner des ombres, tu es sorti du brief. Reviens au parcours : comprendre, comparer, s'inscrire.

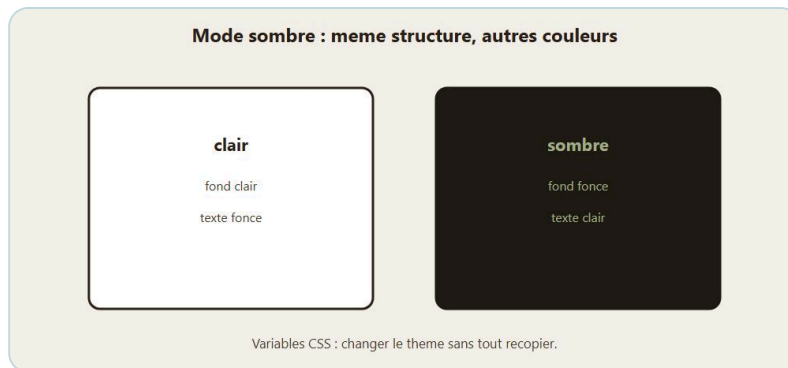
Bonus

Pastille "Places limitees" dans une carte tarif (la carte en `position: relative` si besoin). Ou un second theme via classe, comme a l'atelier variables. Optionnel.

A toi

Livre `landing.html` + `landing.css`. C'est valide quand tu oserais l'envoyer a un ami pour avis. Ecris ensuite trois retours possibles que tu redoutes (trop long, prix flou, formulaire penible) et corrige le plus probable avant d'envoyer vraiment. Chez DanielCraft, oser envoyer bat peaufiner sans fin.

Chapitre 17 - Dark mode avec variables



Clair / sombre : memes zones, autres couleurs.

Tu as déjà theme une page via `:root`. Le dark mode, c'est la meme idee, branchee sur la preference systeme (et/ou un choix manuel). Pas besoin de doubler tout ton CSS. Tu redefines les variables. Le reste suit.

Chez DanielCraft, un mode sombre lisible, c'est un confort nocturne, pas un filtre gris illisible. Contraste d'abord, ambiance ensuite. Lea livre souvent les deux themes d'un coup. Max lit ses devis le soir sur le canape : il veut du lisible, pas du "style OLED". Sam montre en classe le bascule systeme des outils developpeur - les eleves voient la page changer sans toucher aux composants.

Ce que ce n'est pas

Ce n'est pas dupliquer toutes les classes (`.carte-dark`, `.bouton-dark` ...). Ce n'est pas inverser seulement le fond et oublier cartes et champs. Ce n'est pas non plus un noir pur `#000` partout "parce que ca fait tech". Et ce n'est surtout pas activer un dark mode alors que ta page a encore des hex en dur partout. Discipline variables d'abord.

Preference systeme

Le systeme de l'utilisateur demande parfois un theme sombre. Toi, tu ecoutes cette demande dans un media query, tu reecris les variables du `:root`, et toute la page bascule. Ensuite, si tu veux, tu ajoutes une classe manuelle qui force clair ou sombre - parce que le choix explicite de l'humain gagne souvent sur le reglage systeme. Les composants ne bougent pas. Seule la palette bouge.

```
:root {
  --fond: #f7f5f0;
  --texte: #1b1b1b;
  --carte: #ffffff;
  --couleur-principale: #1a5f4a;
  --bordure: #ddd;
  color-scheme: light dark;
}

@media (prefers-color-scheme: dark) {
  :root {
    --fond: #121a17;
    --texte: #e8f0eb;
    --carte: #1a2822;
    --couleur-principale: #5dcaa5;
    --bordure: #2c3d34;
  }
}
```

```

    }
}

body {
  background: var(--fond);
  color: var(--texte);
}

```

`color-scheme: light dark` aide le navigateur sur les controles natifs (scrollbars, champs) pour qu'ils ne restent pas "blancs agressifs" en theme sombre. Pas magique partout, mais bon reflexe.

★ A RETENIR

Redefinis les variables, pas les classes. Contraste d'abord, ambiance ensuite. Inputs et ombres doivent suivre le theme aussi.

Bascule manuelle, ombres, formulaires

```

html.theme-dark {
  --fond: #121a17;
  --texte: #e8f0eb;
  --carte: #1a2822;
  --couleur-principale: #5dcaa5;
  --bordure: #2c3d34;
  --ombre: 0 4px 18px rgba(0, 0, 0, 0.45);
}

```

Un bouton "Mode sombre" en JavaScript bascule la classe. Ce livre ne force pas le JS : tu peux tester en collant la classe a la main. Si tu ajoutes le JS plus tard, le CSS est deja pret. Decide si la classe manuelle doit gagner sur `prefers-color-scheme`. Souvent oui.

Une ombre douce en clair devient un halo bizarre ou invisible en sombre : passe aussi `--ombre` en variable. Les photos restent des photos. N'assombris pas une image informative jusqu'a la rendre inutile. Pour les champs :

```

input,
textarea,
select {
  background: var(--carte);
  color: var(--texte);
  border: 1px solid var(--bordure);
}

```

Sans ca, des champs blancs eclatent dans une page sombre.

Contrastes, pas ambiance

Le dark mode "instagrammable" trompe. Vert menthe pale sur fond tres sombre, magnifique en capture, illisible en paragraphe. Lea l'a appris chez un client. Elle a eclairci `--texte`, teinte le fond, corrige les inputs. Le client a prefere la version lisible. Max teste le soir sur le canape. Sam chronometre : "lis ce paragraphe a voix haute en theme sombre". Si tu butes, le contraste est trop faible.

Vérifie aussi les CTA et les liens. Un bouton qui disparaît en sombre, c'est pire qu'un fond un peu trop gris. Chez DanielCraft, lisibilité d'abord, ambiance ensuite. Un presque noir teinté (#121a17) est souvent plus doux qu'un noir pur sur OLED.



Exemple : un thème sombre sans tout recopier.

Petite histoire

Lea avait livré un dark mode "instagrammable" : vert menthe pale sur fond très sombre. Elle a éclairci `--texte`, foncé légèrement le fond teinté, et corrigé les inputs. Max a testé le thème sombre de sa page artisan : le bouton devis restait vert clair sur vert clair - oubli de variable sur le bouton. Sam a transformé l'incident en exo de contraste.

⚠ ATTENTION

Contrastes insuffisants "parce que c'est style". Ou CTA qui disparaît. Ou champs blancs oubliés dans une page sombre. Vérifie texte, liens, boutons, labels, inputs.

Erreur classique

Dupliquer toutes les classes en `.dark`. Activer le mode sombre alors que des hex trainent partout. Hero qui dépend du `--fond` sans test des deux thèmes.

En vrai

Reprends le mini-projet ou l'atelier variables. Ajoute le media `prefers-color-scheme: dark`. Bascule le thème OS (ou l'émulation dans les outils développeur). Regarde formulaire, cartes, header. Ajoute ensuite `html.theme-dark` comme override manuel et teste les deux chemins.

A toi

Active le dark mode par variables sur une page complète (pas un seul bloc). Checklist : body, cartes, bordures, bouton, inputs, ombres. Note une couleur que tu as du corriger pour le contraste. C'est bon signe : tu regardes vraiment. Chez DanielCraft, ce regard-là compte plus qu'un thème "wow" illisible.

Chapitre 18 - Perf CSS legeres : aller vite sans obsession



CSS utile, images legeres, ressenti rapide.

La performance, ce n'est pas seulement JavaScript et images. Le CSS peut aussi ralentir ou faire "sauter" une page. Bonne nouvelle : pour un site vitrine simple, quelques hygiene habits suffisent. Pas besoin de devenir ingenieur navigateur.

Chez DanielCraft, on vise "snappy et propre" : chargement raisonnable, peu de surprise visuelle, CSS maintenable. Lea coupe une image hero avant d'optimiser une media query. Max se fiche des scores Lighthouse : il veut que sa page devis s'ouvre vite sur 4G. Sam apprend a ses eleves a mesurer avant de micro-optimiser. Trois postures, meme ordre : d'abord le gros, ensuite le detail.

Ce que ce n'est pas

Ce n'est pas courir apres 100/100 sur une page d'atelier. Ce n'est pas inliner du "critical CSS" obligatoire a ton niveau. Ce n'est pas non plus ajouter une lib CSS entiere pour deux boutons. Et ce n'est surtout pas micro-optimiser le CSS pendant qu'un PNG de logo fait 2 Mo. La perf honnete commence par le poids evident.

Ordre d'attaque

Tu ouvres l'onglet Reseau. Tu vois le poids total et le poids images. Tu listes CSS, images, polices. Tu attaques le plus lourd. Souvent, ce n'est pas ta feuille de style : c'est le hero. Ensuite tu nettoies les regles mortes, tu limites les fonts, tu reserves l'espace des media pour eviter les sauts. La page respire. Tes visiteurs ne savent pas pourquoi - ils restent.

Chez DanielCraft, on attaque dans cet ordre : images, polices, CSS mort, animations permanentes, micro-selecteurs. Pas l'inverse. Si tu n'as que trente minutes, fais uniquement l'image la plus lourde. Un gros gain visible bat dix micro-optimisations invisibles.

★ A RETENIR

Mesure avant de micro-optimiser. Images d'abord, puis polices, puis CSS mort. Un gros gain visible bat dix tweaks invisibles.

Hygiene CSS et selecteurs

Chaque regle que tu n'utilises plus reste un poids mental (et parfois reseau). Quand tu termines un atelier, supprime les essais commentes sur vingt lignes. Evite les selecteurs ultra longs : fragiles, et le navigateur prefere des classes simples sur les elements concernees.

```
/* Fragile */
body div.main section.cartes article.carte p.prix span {}

/* Clair */
.carte-prix {}
```

Animer `width`, `height`, `top`, `left` peut couter plus cher qu'animer `transform` et `opacity`. Pour les hovers de cartes, tu le sais : `translateY` + ombre. Ne recalcule pas toute la page a chaque hover.

Images, fonts, stabilite

Compresse. Redimensionne a la taille utile. `width / height` ou `aspect-ratio` pour reserver l'espace (moins de layout shift). `srcset` quand tu as plusieurs fichiers. Une landing avec un hero 3 Mo restera lente quoi que tu fasses en CSS.

```
img,
video {
  max-width: 100%;
  height: auto;
}

.carte img {
  aspect-ratio: 4 / 3;
  object-fit: cover;
}
```

Charger cinq familles Google Fonts pour trois titres ralentit. Une ou deux familles suffisent. `font-display: swap` (quand tu charges une webfont) evite le texte invisible trop longtemps. Sur beaucoup de pages de ce livre, une stack systeme soignee reste honnete et rapide.

Petite histoire

Lea a passe une apres-midi a "optimiser" des transitions pendant qu'une image Instagram exportee a pleine resolution plombait la home. Elle a recomprime : -70 % sur le fichier, page transformee. Max a demande a son neveu pourquoi le site "saccadait" : une image sans hauteur reservee faisait sauter le bouton devis. Sam a affiche le panneau Reseau en projecteur : silence dans la classe, puis "ah".

⚠ ATTENTION

Dix animations permanentes, ombres a dix couches, meme image hero en fond CSS et en balise `img` : tu alourdis sans gagner. Et non, les variables CSS ne "ralentissent" pas le site.

Erreur classique

Micro-optimiser le CSS pendant qu'un PNG de 3 Mo plombe la home. Croire que les variables ralentissent. Charger cinq fonts pour trois titres. Ou dix animations permanentes "pour faire vivant" : souvent elles coutent plus qu'elles n'apportent.

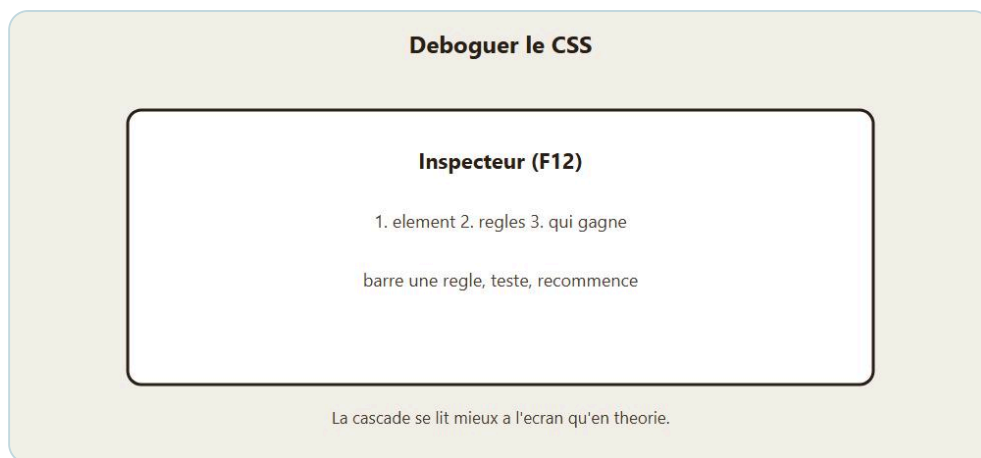
En vrai

Sur ta landing, liste les fichiers. Note le plus lourd. Attaque celui-la. Recomprime l'image. Supprime une font inutile. Relance. Sens la difference. Cherche ensuite une classe CSS orpheline : si elle n'existe plus dans le HTML, vire-la. Si tu as trente minutes seulement, ignore le CSS mort et fais uniquement l'image. Lea a perdu une apres-midi sur des transitions pendant qu'un hero Instagram plombait tout. Max a gagne dix secondes de perception en compressant juste le hero. Sam affiche le waterfall Reseau : les eleves voient le "gros" avant le "fin".

A toi

Fais un "pass perf" de 30 minutes sur une page : (1) image la plus lourde recompressée, (2) CSS nettoye d'essais, (3) verification `max-width` sur media, (4) une seule famille de police principale. Ecris le poids avant/apres de l'image. Objectif : -50 % sur cette image, ou mieux. Garde le chiffre. C'est ta preuve, pas un sentiment.

Chapitre 19 - Debug CSS : outils, outline, hypotheses



F12 : element, regles, qui gagne.

Ca ne s'affiche pas comme tu veux. Normal. Le debug CSS est un metier de detective : hypothese, test, observation. Pas de panique aleatoire ni de **!important** en rafale.

Chez DanielCraft, on debug comme on range un atelier : une cause a la fois, de la lumiere partout. Lea ouvre l'inspecteur avant de reecrire un composant. Max appelle son neveu quand ca "debord" - et le neveu, s'il a lu ce chapitre, demande d'abord quelle hypothese. Sam note au tableau : "une cause, un test". Les eleves qui suivent ca avancent deux fois plus vite.

Ce que ce n'est pas

Ce n'est pas ajouter des marges negatives au hasard pour "ramener" un bloc. Ce n'est pas creer une classe **.fix** avec **!important** sur tout. Ce n'est pas modifier le HTML structure pour compenser un CSS confus sans comprendre. Et ce n'est pas changer cinq proprietes d'un coup en esperant que ca se remette. Le tire a l'aveugle fatigue plus que la methode.

Methode en cinq gestes

Tu reproduis le bug a une largeur precise. Tu inspectes l'element. Tu lis qui gagne sur la propriete concerne. Tu poses une hypothese unique a voix haute : "je pense que la carte debord parce que l'image n'a pas **max-width: 100%**". Tu testes uniquement ca. Si faux, nouvelle hypothese. Si vrai, tu nettoies les outlines de debug et tu continues. Le muscle, c'est l'hypothese, pas le genie nocturne.

Sam force ses eleves a dire la phrase avant de toucher le CSS. Si tu ne peux pas finir la phrase, tu n'as pas encore d'hypothese - tu as de la panique. Lea le fait seule devant l'inspecteur. Max l'a appris apres une heure de marges negatives. Cinq minutes de methode battent quarante minutes de tire a l'aveugle.

★ A RETENIR

Reproduire, inspecter, lire qui gagne, une hypothese a voix haute, un seul test. Le muscle, c'est l'hypothese unique.

L'inspecteur et l'outline

F12 (ou clic droit, Inspector). Onglet Styles : regles actives, regles barrees. Onglet Calcule / Computed : valeur finale. Boite (margin/padding) visualisee. Si une couleur "ne change pas", regarde qui gagne (specificite, ordre, inline). Le chapitre cascade t'a prepare : l'inspecteur le montre en vrai.

Quand tu ne vois plus qui est qui :

```
* {
  outline: 1px solid tomato;
}
```

Ou plus cible :

```
.layout > * {
  outline: 2px dashed #1a5f4a;
}
```

Tu vois les boites. Tu vois les debordements. Tu enlèves ensuite. C'est un projecteur, pas une deco. Active aussi le mode "montrer les grilles Flex/Grid" de l'inspecteur sur le conteneur.

♦ ASTUCE

Outline temporaire sur `.layout > *` pendant cinq minutes. Tu vois les boites. Tu enlèves apres. Projecteur, pas deco.

Checklist des pieges frequents

`max-width` manquant sur image ou iframe. Parent en Grid/Flex mais enfant avec largeur fixe trop grande. `overflow` qui cache focus, ombre ou menu. Mauvaise `grid-area` (typo dans le nom). Media query jamais atteinte (breakpoint ou syntaxe). Classe mal orthographiee dans le HTML (`.carte` vs `cart`). Fichier CSS non lie, mauvais chemin, ou ancien cache (rechargement force).

```
* {
  box-sizing: border-box;
}
```

Souvent en debut de feuille. Sinon `width: 100%` + padding peut faire deborder. Debug Grid : colorie les zones, verifie que chaque ligne de `grid-template-areas` a le meme nombre de cellules, regarde si un enfant sans area se place dans la premiere case libre. Debug Flex : `flex-wrap` oublie, parfois `min-width: 0` sur un enfant qui refuse de retrecir. Debug focus : cherche `outline: none` ou un overflow qui coupe, Tab en regardant `:focus`.

Petite histoire

Lea a perdu quarante minutes sur un menu qui "mangeait" le focus : un `overflow: hidden` sur le header. Outline partout, hypothese, correctif, cinq minutes. Max avait un bouton qui sortait de l'ecran : image sans `max-width`. Sam a casse volontairement une page en cours (faute de `grid-area`) et a chronometre le debug methodique versus le debug panique. Methode : gagnante, sans drama.

⚠ ATTENTION

Corriger le symptôme (marge negative, taille magique) au lieu de la cause. Empiler les **!important** jusqu'a ne plus savoir qui commande. Quand tu ne sais plus, repars de l'inspecteur et d'une seule propriete.

Erreur classique

Desactiver le cache mentalement ("ca marche chez moi") sans hard refresh. Changer cinq proprietes d'un coup. Modifier le HTML structure pour compenser un CSS confus sans comprendre.

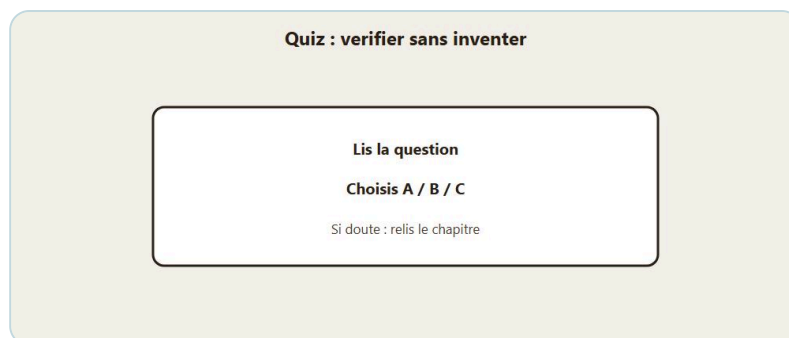
En vrai

Casse volontairement une page qui marche : retire **max-width** sur une image, faute une **grid-area**, baisse un breakpoint. Puis debug avec la methode en cinq minutes : reproduire, inspecter, lire qui gagne, hypothese unique, tester. Remets propre. Fais la meme chose sur un conflit de couleur. Tu entraines le reflexe.

A toi

Prends un bug reel sur ton mini-projet (meme petit). Ecris l'hypothese en une phrase. Corrige. Ajoute en commentaire CSS une ligne **/ fix: image max-width - debord mobile /** le temps de t'en souvenir, puis efface le commentaire si trop bruyant. Garde surtout la phrase d'hypothese dans ton carnet. C'est ca, le vrai outil.

Quiz final



Verifier ce que tu as compris.

Pas de piege. Relis une fois si besoin. L'idée : verifier que tu peux avancer sans paniquer. Chez DanielCraft, un quiz sert a reperer les trous, pas a humilier. Lea note ses erreurs. Max refait un atelier Grid. Sam l'utilise en fin de module. Toi, tu coches sans tricher.

Une premiere passe honnete vaut mieux qu'un score maquille. Si tu bloques, marque et continue. Le but, c'est la carte des chapitres a rouvrir. Fais le quiz, note ton score, rejoue dans une semaine apres avoir touche une vraie page. Le vrai progres se voit a la deuxieme passe.

♦ ASTUCE

Entoure deux questions ratees des que tu as fini. Rouvre le chapitre correspondant le soir meme, sans attendre "plus tard".

Questions

1. 1. Pourquoi preferer `header`, `main`, `nav`, `footer` a une pile de `div` anonymes ?

- A) Parce que ca change automatiquement les couleurs
- B) Parce que ca donne un sens clair a la structure (humains et machines)
- C) Parce que Flexbox refuse les `div`

1. 2. A specificite egale, quelle regle gagne en general ?

- A) La premiere du fichier
- B) La derniere du fichier
- C) Celle ecrite en anglais

1. 3. Ou declare-t-on souvent les variables CSS globales ?

- A) Dans `:root`
- B) Uniquement dans chaque balise `<img>`
- C) Dans le fichier `robots.txt`

1. 4. `display: grid` sert surtout a :

- A) Remplacer HTML par CSS
- B) Poser un layout en lignes et colonnes

- C) Compresser les images

1. 5. Dans `grid-template-areas`, chaque ligne du dessin doit :

- A) Avoir le meme nombre de cellules
- B) Commencer par un `#`
- C) Etre vide

1. 6. Pour un menu horizontal logo + liens, le reflexe le plus naturel est souvent :

- A) Flexbox
- B) Une table HTML a 12 colonnes
- C) `position: fixed` sur chaque lien

1. 7. Pour des vignettes produit de meme taille, un bon duo est :

- A) `object-fit: cover` dans un cadre fixe / `aspect-ratio`
- B) `object-fit: fill` toujours, meme si ca ecrase
- C) Largeur fixe 2000px sur mobile

1. 8. Un vrai `label` de formulaire, c'est surtout :

- A) Un placeholder qui disparaît
- B) Un texte relie au champ (souvent via `for` / `id`)
- C) Une image decorative

1. 9. Ou placer en general la propriete `transition` pour un aller-retour doux ?

- A) Uniquement dans `:hover`
- B) Sur l'etat de base (ex: `.bouton`)
- C) Dans la balise `<title>`

1. 10. `:focus-visible` aide surtout a :

- A) Masquer le focus pour toujours
- B) Montrer clairement le focus clavier sans bruler tout le design
- C) Activer le mode sombre

1. 11. Pour un dark mode maintenable, le meilleur levier est :

- A) Dupliquer toutes les classes en `.dark`
- B) Redefinir les variables CSS (media et/ou classe)
- C) Mettre `filter: invert(1)` sur `body` et prier

1. 12. Bonne pratique generale :

- A) `!important` partout pour aller plus vite
- B) Variables + HTML semantique + test clavier regulier
- C) Tout positionner en `absolute`

Corriges

1-B, 2-B, 3-A, 4-B, 5-A, 6-A, 7-A, 8-B, 9-B, 10-B, 11-B, 12-B.

Si tu as 9/12 ou plus : tu es prêt pour des pages réelles plus propres. Sinon, relis les chapitres liés (sémantique, cascade, variables, Grid, images, formulaires, a11y, dark mode), sans dramatiser.

Petite histoire

Lea a raté la question focus. Elle a tapé sa landing le soir même et a corrigé le contour. Max a confondu Flex et Grid : il a relu le chapitre 7 dix minutes. Sam affiche le score moyen : "on revoit variables et labels demain". Trois réactions, même message : le quiz est une carte. Chez DanielCraft, on aime ces cartes.

★ A RETENIR

Le quiz n'est pas un examen : c'est une carte des trous à boucher. Le vrai score, c'est la deuxième passe après une vraie page.

Mini debrief

Entoure deux questions ratées. Rouvre le chapitre correspondant. Refais l'exercice "A toi" de ce chapitre.

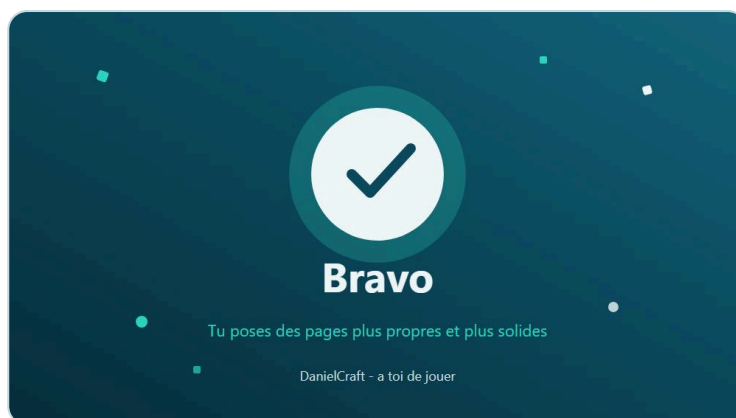
En vrai

Ouvre ta dernière page. Vérifie une chose liée à une question ratée : un label, une variable, un focus, une grille. Cinq minutes. Un trou à la fois.

A toi

Écris tes trois questions ratées. À côté, une phrase "ce que je confonds". Dans sept jours, rejoue seulement ces trois. Le vrai score, c'est la deuxième passe.

Bravo



Bravo. Tu sais poser des pages plus solides.

Sérieusement : bravo.

Tu as fini HTML et CSS - La suite. Pas juste survole. Tu as vu la sémantique, la cascade, les variables, Grid (bases et mise en page), Flex vs Grid, les images modernes, les formulaires stylés, les transitions légères, l'accessibilité suite, un mini-projet, des ateliers, le dark mode, un peu de perf, et du debug méthodique.

C'est exactement comme ça que ça marche : lire, essayer, planter un peu, corriger, continuer. Chez DanielCraft, on ne célèbre pas la page parfaite du premier jet. On célèbre la page que tu as ramené propre après les hypothèses.

Lea s'en sert pour livrer plus vite sans baisser la qualité. Max s'en sert pour une page artisan claire sur téléphone. Sam s'en sert pour expliquer sans jargon opaque. Toi, tu t'en sers demain matin sur ta propre page. Trois métiers, un même ordre mental : sens, marque, plan, détail, vérification.

Ce que tu sais faire maintenant

Tu sais poser une page avec un vrai plan (Grid) et une vraie cohérence de marque (variables). Tu comprends mieux qui gagne en CSS. Tu choisis Flex ou Grid avec intention. Tu cadres des images sans les massacrer. Tu styles un formulaire pour qu'on ait envie de l'utiliser. Tu ajoutes du mouvement sans transformer le site en fête foraine. Tu penses clavier, contraste, labels. Tu bascules un thème sombre sans dupliquer tout le CSS. Tu debugues avec des hypothèses, pas avec de la panique.

Les bases du premier livre sont toujours là. La suite, c'est elles en mieux rangées, plus solides, plus accueillantes.

Devant une page blanche, tu ne commences plus par "quelle lib ?". Tu commences par le sens (HTML), la marque (`:root`), le plan (Grid/Flex), le détail utile (images, formulaire, transitions), puis tu vérifies humains et machines (a11y, perf, debug). Cet ordre te sortira de beaucoup de chaos. Garde-le.

★ A RETENIR

Sens (HTML), marque (`:root`), plan (Grid/Flex), détail utile, puis vérification humaine. Cet ordre te sortira de beaucoup de chaos.

Petite mission (si tu veux)

Reprends ta landing ou ta page produit. Ameliore juste trois trucs : un theme coherent 100 % variables (clair + sombre), une grille de cartes en `auto-fit` + `minmax`, un passage Tab sans accroc + focus visible. Montre-la a quelqu'un. Demande ce qui est clair en cinq secondes. Ecoute. Ajuste. Ce feedback humain vaut plus qu'un score automatique.

Ce que ce n'est pas

Ce n'est pas la fin de ton apprentissage web. Ce n'est pas non plus "tu dois enchaîne JavaScript ce soir". Tu as le droit de souffler. Une competence solide se digere. Si tu forces la suite fatigue, tu retiendras moins. Si tu laisses reposer une semaine puis tu reconstruis une petite page de zero, tu verras ce qui est vraiment entre.

Petite scene DanielCraft

Lea ouvre une page client "urgente". Elle pose d'abord les variables et la grille, pas les ombres. Max teste sa page devis sur telephone avant d'envoyer le lien. Sam casse volontairement une `grid-area` en classe, puis chronometre le debug methodique. Trois gestes, une meme posture : hypothese avant panique. C'est le vrai bravo.

En vrai

Ouvre ta derniere page. Bascule le theme OS (ou emule le dark mode). Tab une fois sur le formulaire. Regarde le poids de l'image hero dans Reseau. Trois gestes, trois minutes. Note ce qui frotte. C'est ta feuille de route pour cette semaine - pas une to-do heroique.

La suite, quand tu seras chaud

JavaScript, pour rendre la page un peu vivante (formulaires qui verifient, menus, contenus qui se chargent...). Ou un vrai petit site multi-pages en ligne. Ou le livre JavaScript chez DanielCraft, si tu veux un chemin guide. Ou simplement publier ta landing et la montrer. Publier, c'est un super bravo aussi.

Mais pas ce soir si tu es fatigue. Repose-toi. Tu l'as merite.

A toi

Ecris en trois lignes : la page que tu vas ameliorer, le premier correctif (variables, grille, ou focus), et la personne a qui tu vas la montrer. Puis ouvre l'editeur dix minutes. Pas demain. Maintenant.

Encore bravo. Et a bientot pour la suite.

Tu as les briques. Maintenant, construis.